

INFORMATIQUE • TLETI

MODULE 3 : SYSTÈMES D'INFORMATION ET BASES DE DONNÉES

LEÇON N12

MODÉLISATION STRUCTURELLE DES SYSTÈMES D'INFORMATION AVEC UML (DIAGRAMMES DE CLASSES ET D'OBJETS)

DURÉE CONSEILLÉE : 12 H

DANS CETTE LEÇON

Cette leçon initie les élèves à la modélisation structurelle des systèmes d'information avec UML. Ils apprennent à représenter classes et objets, à matérialiser associations, multiplicités, héritage, agrégation et composition, puis à produire un diagramme de classes complet (au plus cinq classes) et le diagramme d'objets correspondant.

- À la fin de cette leçon, je sais définir UML et citer le rôle des diagrammes structurels (classes, objets) dans la conception d'un système d'information.
- À la fin de cette leçon, je sais représenter une classe en UML avec son nom, ses attributs et ses méthodes.
- À la fin de cette leçon, je sais coder la visibilité d'un élément (+, -, #, ~) et préciser les paramètres et le type de retour d'une méthode.
- À la fin de cette leçon, je sais représenter un objet (instance) dans le formalisme UML et le relier à sa classe.
- À la fin de cette leçon, je sais matérialiser une association entre classes et y annoter les multiplicités (1, 0..1, 1..*, *).
- À la fin de cette leçon, je sais représenter une relation d'héritage et identifier les attributs et méthodes hérités.



TÉLÉCHARGE ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA illimité, fiches PDF — onglet OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir

**Signé • OnBuch+**

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

- | | |
|--|----------------------------------|
| 1. UML et les diagrammes structurels | un diagramme d'objets |
| 2. Représentation d'une classe | 7. Activité d'intégration |
| 3. Représentation d'un objet | 8. Méthodes et exercices résolus |
| 4. Associations et multiplicités | 9. Exercices |
| 5. Héritage, agrégation et composition | 10. Corrigés des exercices |
| 6. Produire un diagramme de classes et | 11. Fiche bilan |

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais définir UML et citer le rôle des diagrammes structurels (classes, objets) dans la conception d'un système d'information.
- À la fin de cette leçon, je sais représenter une classe en UML avec son nom, ses attributs et ses méthodes.
- À la fin de cette leçon, je sais coder la visibilité d'un élément (+, -, #, ~) et préciser les paramètres et le type de retour d'une méthode.
- À la fin de cette leçon, je sais représenter un objet (instance) dans le formalisme UML et le relier à sa classe.
- À la fin de cette leçon, je sais matérialiser une association entre classes et y annoter les multiplicités (1, 0..1, 1..*, *).
- À la fin de cette leçon, je sais représenter une relation d'héritage et identifier les attributs et méthodes hérités.
- À la fin de cette leçon, je sais distinguer et dessiner une agrégation (losange vide) et une composition (losange plein).
- À la fin de cette leçon, je sais produire un diagramme de classes d'au plus cinq classes à partir d'un énoncé de gestion réel.
- À la fin de cette leçon, je sais produire le diagramme d'objets correspondant à un diagramme de classes à un instant donné.

PRÉREQUIS

- Notion de système d'information et d'acteurs (Module 3, début d'année)

- Notion d'entité, d'attribut et de relation vus en Première (modèle entité-association)
- Vocabulaire de base de la programmation : variable, type de données, fonction avec paramètres et valeur de retour
- Notion de base de données et d'enregistrement (Première TI)

1 UML ET LES DIAGRAMMES STRUCTURELS

1.1 Situation d'accroche : la bibliothèque du lycée bilingue de Yaoundé

La direction du lycée bilingue de Yaoundé vient de prendre une décision stratégique : remplacer le registre manuscrit du documentaliste par un véritable logiciel de gestion de bibliothèque. Le système devra gérer les **livres** (titre, auteur, cote, état), les **élèves** (nom, classe, matricule), les **emprunts** (date de sortie, date de retour prévue) et les **amendes** en cas de retard.

Le proviseur, pressé, demande au jeune informaticien recruté : « Quand commences-tu à programmer ? ». Celui-ci répond : « Pas encore. D'abord, j'établis le plan. » Comme un architecte ne coule jamais le béton avant d'avoir tracé chaque mur sur un plan, l'informaticien n'écrit aucune ligne de code avant d'avoir modélisé le futur système.

Problématique : comment représenter les « choses » du système (un livre, un élève, un emprunt), leurs caractéristiques et leurs liens, de façon universelle, précise et sans ambiguïté, avant même de choisir un langage de programmation ?

C'est exactement le rôle des **diagrammes de classes** et des **diagrammes d'objets** d'UML, le langage de modélisation utilisé par les entreprises du monde entier, des startups de la Silicon Valley aux sociétés de développement de Douala et de Yaoundé.

1.2 Qu'est-ce qu'un système d'information et pourquoi le modéliser ?

Intuition. Un **système d'information** (SI) est l'ensemble organisé de ressources (humaines, matérielles, logicielles, documentaires) qui permet de collecter, stocker, traiter et diffuser l'information dans une organisation. Dans notre lycée, le SI de la bibliothèque comprend le documentaliste (ressource humaine), l'ordinateur et le logiciel (ressources matérielles et logicielles), et les données sur les livres et les emprunts (ressources documentaires).

Pourquoi ne pas programmer directement ? Parce qu'un logiciel réel est un objet complexe : des dizaines de tables en base de données, des milliers de lignes de code, plusieurs développeurs en parallèle. Sans plan commun, chacun interprète le projet à sa manière, et le résultat est un désastre : données incohérentes, fonctionnalités oubliées, budget explosé. Les études sur les projets informatiques montrent qu'une erreur de conception détectée au stade du code coûte dix à cent fois plus cher à corriger que si elle avait été repérée sur le modèle.

DÉFINITION Modéliser un système

Modéliser un système, c'est en construire une représentation simplifiée et abstraite qui met en évidence les aspects essentiels (entités manipulées, propriétés, relations) en écartant les détails d'implémentation. Le modèle sert de support de communication entre le client, l'analyste et le programmeur, et de référence tout au long du cycle de vie du logiciel.

L'analogie avec le bâtiment est parlante : le plan de l'architecte n'est pas la maison, mais il permet au maçon, au plombier et à l'électricien de travailler ensemble sans se tromper. De même,

le modèle UML n'est pas le logiciel, mais il guide le développeur PHP, l'administrateur de base de données et le chef de projet.

1.3 UML : un langage graphique normalisé

Le problème historique. Dans les années 1980–1990, chaque méthode d'analyse (OMT, Booch, OOSE...) proposait ses propres notations : un rectangle ici, un nuage là... Deux informaticiens formés à des méthodes différentes ne se comprenaient pas. Il fallait un langage commun, comme l'anglais pour l'aviation internationale.

DÉFINITION UML

UML (Unified Modeling Language, langage de modélisation unifié) est un langage de modélisation graphique normalisé destiné à spécifier, visualiser, construire et documenter les systèmes logiciels. UML est :

- graphique : on dessine des boîtes, des flèches et des symboles plutôt que d'écrire de longs textes ;
- normalisé : sa grammaire (symboles et sémantique) est définie par l'**OMG** (Object Management Group) ; un diagramme dessiné à Yaoundé se lit de la même façon à Montréal ou à Tokyo ;
- indépendant des langages de programmation : un même diagramme de classes peut être implémenté en PHP, Java, Python ou traduit en tables SQL ;
- orienté objet : il décrit le monde comme un ensemble d'objets dotés d'identité, de caractéristiques (attributs) et de comportements (méthodes).

ATTENTION UML n'est ni une méthode ni un langage de programmation

Deux confusions classiques à éliminer dès maintenant :

- UML n'est pas un langage de programmation : on ne peut pas « exécuter » un diagramme UML. Il sert à concevoir, pas à coder.
- UML n'est pas une méthode de développement : il ne prescrit pas quand ni dans quel ordre produire les diagrammes ; il fournit uniquement les notations (la syntaxe graphique).

1.4 Les deux grandes familles de diagrammes UML

UML 2.5 définit 14 types de diagrammes, rangés en deux familles selon la question à laquelle ils répondent.

DÉFINITION Diagramme structurel (statique)

Un **diagramme structurel** (ou statique) représente les éléments constitutifs du système et leurs relations, indépendamment du temps. Il répond à la question : « De quoi le système est-il fait ? ». Il décrit les entités du domaine (livres, élèves, emprunts), leurs propriétés et leurs liens, comme un plan d'architecte décrit les pièces d'une maison.

DÉFINITION Diagramme comportemental (dynamique)

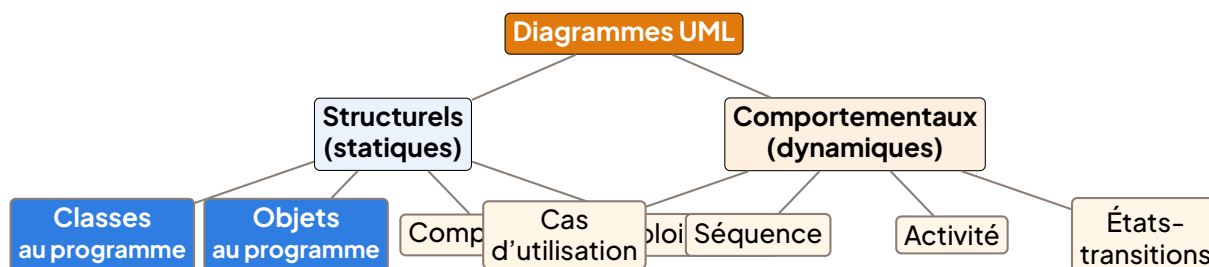
Un **diagramme comportemental** (ou dynamique) représente ce qui se passe dans le système au cours du temps : scénarios d'utilisation, enchaînement d'actions, échanges de messages. Il répond à la question : « Comment le système se comporte-t-il ? ». Exemple : le diagramme de cas d'utilisation montre que l'élève peut emprunter un livre et que le documentaliste peut enregistrer un retour.

PROPRIÉTÉ Classification des diagrammes UML

Les diagrammes UML se répartissent en deux familles :

- Diagrammes structurels : diagramme de **classes**, diagramme d'**objets**, diagramme de composants, de déploiement, de paquetages, de structure composite ;
- Diagrammes comportementaux : diagrammes de cas d'utilisation, de séquence, d'activité, d'états-transitions, de communication, de temporisation, d'aperçu d'interaction.

Au programme de Terminale TI, seuls le diagramme de classes et le diagramme d'objets sont exigibles.



Classification des diagrammes UML. Les deux diagrammes exigibles au Baccalauréat TI (diagramme de classes et diagramme d'objets) sont mis en évidence : ils appartiennent à la famille structurelle.

LESAVAIS-TU ? UML, une norme mondiale

UML a été normalisé par l'OMG en 1997, à partir des travaux de trois informaticiens américains : Grady Booch, Ivar Jacobson et James Rumbaugh, surnommés les « trois amis ». La version actuelle, UML 2.5, compte 14 types de diagrammes officiels. Bonne nouvelle pour toi : seuls deux d'entre eux — le diagramme de classes et le diagramme d'objets — sont exigibles au Baccalauréat TI. Les grandes structures camerounaises (CENADI, ANTIC, éditeurs de logiciels bancaires, opérateurs télécoms) utilisent UML dans leurs cahiers des charges pour dialoguer avec leurs prestataires.

1.5 Le diagramme de classes : la structure générale du système

Intuition. Reprends la bibliothèque. Il existe potentiellement des milliers de livres, mais ils partagent tous la même structure : chacun possède un titre, un auteur, une cote, un état (disponible ou emprunté). Plutôt que de dessiner chaque livre un par un, on dessine le moule commun à tous les livres : la **classe** Livre. Le diagramme de classes est le plan de tous ces moules et des liens qui les unissent.

DÉFINITION Diagramme de classes

Le **diagramme de classes** est le diagramme structurel central d'UML. Il décrit la structure statique du système : les classes (types d'entités du domaine), leurs attributs (propriétés), leurs méthodes (comportements) et les relations entre classes (associations, héritage, agrégation, composition). Il ne montre aucun cas particulier : il décrit la structure générale et permanente du système.

Dans notre exemple, le diagramme de classes dira : « un Livre possède un titre, un auteur et une cote ; un Emprunt possède une date de sortie et une date de retour prévue ; un emprunt concerne exactement un livre ». Cette description reste vraie aujourd'hui, demain et dans dix ans : elle est statique.

1.6 Le diagramme d'objets : une photographie à un instant t

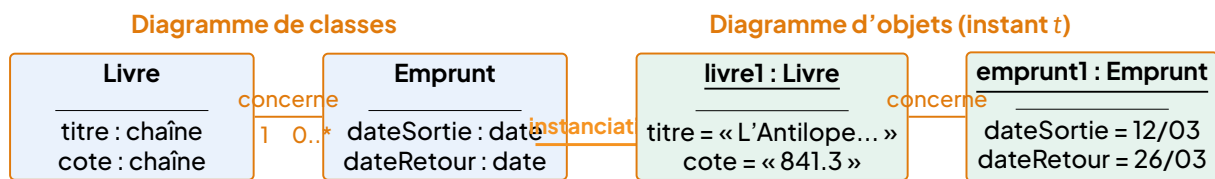
Intuition. Maintenant, ouvre le registre de la bibliothèque ce matin à 10 h : le livre « L'Antilope et la chasse », cote 841.3, est sorti avec l'élève Amina le 12 mars. Ce ne sont plus des moules, mais des exemplaires concrets : des **objets**. Le diagramme d'objets est la photographie du système à un instant précis.

DÉFINITION Diagramme d'objets

Le **diagramme d'objets** représente des instances concrètes des classes (les objets) et les liens effectifs entre eux, à un instant donné. Chaque objet est noté sous la forme `nomObjet` : `NomClasse`, avec le nom souligné selon la convention UML, par exemple `livre1 : Livre`.

PROPRIÉTÉ Lien entre diagramme d'objets et diagramme de classes

Le diagramme d'objets est une instance du diagramme de classes à un instant t : chaque objet du diagramme d'objets est l'instanciation d'une classe du diagramme de classes, et chaque lien entre objets respecte une association déclarée entre leurs classes. C'est une convention fondamentale d'UML. Conséquence pratique : un diagramme d'objets ne peut jamais montrer un objet d'une classe inexistante ni un lien interdit par le diagramme de classes.



À gauche, le diagramme de classes décrit la structure générale (les moules).

À droite, le diagramme d'objets photographie des instances concrètes à un instant t : chaque objet est obtenu par instanciation d'une classe.

ATTENTION Erreur fréquente : confondre classe et objet

La confusion la plus classique au Baccalauréat est de mélanger les deux diagrammes :

- le diagramme de classes décrit la structure générale : types abstraits (Livre, Emprunt), valables en permanence, sans aucune valeur concrète ;
- le diagramme d'objets décrit des cas particuliers concrets : livre1, emprunt1, avec des valeurs réelles, à un instant précis.

Indices pour ne plus se tromper : dans un diagramme d'objets, les noms sont soulignés et écrits `nomObjet : Classe`, et les attributs ont des valeurs (`cote = « 841.3 »`). Dans un diagramme de classes, les noms ne sont jamais soulignés et les attributs n'ont que des types (`cote : chaîne`). Si l'énoncé parle d'un livre précis emprunté un jour précis, c'est un diagramme d'objets ; s'il parle des livres en général, c'est un diagramme de classes.

1.7 Du modèle vers la réalisation : base de données et code

Le diagramme de classes n'est pas une fin en soi : c'est la source à partir de laquelle on construit le système réel. La traduction suit des règles simples que tu retrouveras dans la suite du module (Module 3, Leçon 13 sur les bases de données) :

- chaque **classe** devient généralement une **table** dans la base de données relationnelle (ou une classe dans le langage de programmation) ;
- chaque **attribut** devient une **colonne** (champ) de cette table ;

- chaque **association** devient une **clé étrangère** ou une table de liaison ;
- chaque **objet** devient une **ligne** (un enregistrement) de la table.

Voici, à titre d'anticipation, la traduction en SQL (MySQL/MariaDB, standard WampServer) de notre mini-diagramme de classes :

```
-- Table issue de la classe Livre
CREATE TABLE Livre (
  idLivre INT PRIMARY KEY AUTO_INCREMENT,
  titre VARCHAR(150) NOT NULL,
  cote VARCHAR(10) NOT NULL
);

-- Table issue de la classe Emprunt
-- La cle etrangere materialise l'association "concerne"
CREATE TABLE Emprunt (
  idEmprunt INT PRIMARY KEY AUTO_INCREMENT,
  dateSortie DATE NOT NULL,
  dateRetour DATE NOT NULL,
  idLivre INT NOT NULL,
  FOREIGN KEY (idLivre) REFERENCES Livre(idLivre)
);

-- Une ligne = un objet (instance) : le diagramme d'objets devient des donnees
INSERT INTO Livre (titre, cote) VALUES ('L'Antilope et la chasse', '841.3');
```

EXERCICE RÉSOLU 1**Classes ou objets : à toi de trancher**

Énoncé : pour chacune des situations suivantes, indique si sa représentation relève du diagramme de classes ou du diagramme d'objets, en justifiant.

1. « Tout élève du lycée possède un matricule, un nom et une classe. »
2. « Ce matin, l'élève Amina Bekono, matricule 24TI015, a emprunté le livre de cote 841.3. »
3. « Un emprunt concerne un seul livre, mais un livre peut figurer dans plusieurs emprunts au cours de l'année. »

SOLUTION

Solution détaillée :

1. On parle de tout élève, c'est-à-dire de la structure générale, sans valeur concrète : c'est le diagramme de classes (classe Eleve avec attributs matricule, nom, classe).
2. On décrit des individus précis avec des valeurs réelles (Amina Bekono, 24TI015, cote 841.3) à un instant donné (ce matin) : c'est le diagramme d'objets (objets amina : Eleve et livre1 : Livre reliés par un lien).
3. On énonce une règle permanente sur les liens entre types d'entités, avec des multiplicités : c'est le diagramme de classes (association concerne entre Emprunt et Livre).

Réflexe à acquérir : présence de valeurs concrètes et d'un instant précis $\Rightarrow$ objets ; règle générale et permanente $\Rightarrow$ classes.

À RETENIR L'essentiel de la section

- On modélise avant de réaliser pour communiquer sans ambiguïté et détecter les erreurs tôt, quand elles coûtent peu cher.
- UML est un langage de modélisation graphique, normalisé (OMG, 1997) et indépendant des langages de programmation.
- Les diagrammes UML se divisent en structurels (statiques : de quoi le système est-il fait ?) et comportementaux (dynamiques : comment se comporte-t-il ?).
- Le diagramme de classes décrit la structure générale : classes, attributs, méthodes et relations. Le diagramme d'objets est une instance du diagramme de classes à un instant t : noms soulignés, valeurs concrètes.
- Le diagramme de classes se traduit ensuite en tables SQL et en code : c'est le pont entre la conception et la réalisation.

Dans la prochaine section, nous apprendrons à représenter une classe en détail : compartiments, visibilité des attributs et signature des méthodes.

2 REPRÉSENTATION D'UNE CLASSE

Dans la section précédente, nous avons découvert UML comme un langage de modélisation universel et situé les diagrammes structurels — en particulier le diagramme de classes — comme le moyen de représenter la structure statique d'un système d'information. Mais un diagramme de classes, aussi beau soit-il, n'est qu'un assemblage de briques élémentaires : les **classes**. Avant de relier des classes entre elles par des associations ou de l'héritage, il faut donc savoir dessiner correctement une seule classe. C'est précisément l'objet de cette section : apprendre l'alphabet avant d'écrire des phrases.

2.1 Qu'est-ce qu'une classe ?

2.2 De l'observation au modèle

Imagine la bibliothèque de ton lycée. Elle contient des centaines de livres : un roman de Mongo Beti, un manuel de mathématiques, un dictionnaire... Ces livres sont tous différents (titres différents, nombres de pages différents), mais ils partagent les mêmes caractéristiques : chacun possède un titre, un ISBN, un nombre de pages, et chacun peut être emprunté ou rendu. Plutôt que de décrire chaque livre individuellement, on décrit **une seule fois** ce qu'est un livre en général. Cette description commune, c'est la classe.

DÉFINITION Classe

Une **classe** est la description d'un ensemble d'objets qui partagent les mêmes attributs (les données), les mêmes méthodes (les comportements) et la même sémantique (le même rôle dans le système). Une classe est un modèle ou un moule ; chaque objet concret créé à partir de ce moule est appelé une instance de la classe.

Retenons bien le vocabulaire :

- un **attribut** est une donnée qui caractérise l'objet (exemple : le titre d'un livre) ;
- une **méthode** est une action que l'objet sait effectuer ou subir (exemple : emprunter un livre) ;
- un **objet** est une instance concrète de la classe (exemple : ce livre précis, « L'habitant de l'infini », ISBN 978-2-07-036-822-4).

À RETENIR Classe et objet : moule et gâteau

La classe est le moule, l'objet est le gâteau. Avec un seul moule on produit autant de gâteaux qu'on veut : avec une seule classe `Livre`, on représente les 800 livres de la bibliothèque. La classe décrit ce qui est commun ; l'objet porte les valeurs particulières.

2.3 Le rectangle à trois compartiments

UML représente une classe par un **rectangle divisé en trois compartiments horizontaux**, toujours dans le même ordre :

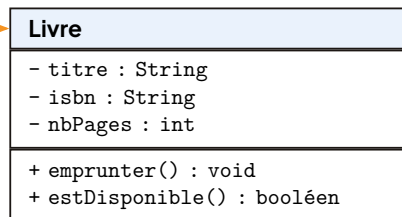
1. le **compartiment du haut** contient le **nom de la classe** ;
2. le **compartiment du milieu** contient la liste des **attributs** ;
3. le **compartiment du bas** contient la liste des **méthodes**.

PROPRIÉTÉ Convention de nommage d'une classe

Le nom d'une classe s'écrit en gras, au singulier, avec la première lettre en majuscule : `Livre`, `Eleve`, `CompteBancaire`. Si le nom comporte plusieurs mots, on les colle en mettant une majuscule à chaque mot (convention dite CamelCase) : `CarteElecteur`, `TransactionMobileMoney`.

Voici notre première classe complète, la classe `Livre`, avec une légende qui identifie chaque compartiment :

Compartiment 1 :
nom de la classe



Compartiment 2 :
attributs (données)

Compartiment 3 :
méthodes (comportements)

Représentation UML de la classe Livre : trois compartiments (nom, attributs, méthodes).

Lisons cette classe : un livre possède trois données (un titre, un ISBN, un nombre de pages) et sait faire deux choses (être emprunté, dire s'il est disponible). Les signes - et + devant chaque élément ne sont pas décoratifs : ce sont les visibilités, que nous allons étudier maintenant.

2.4 Syntaxe d'un attribut

Un attribut ne se résume pas à son nom. UML impose une syntaxe précise qui documente le type de la donnée et, éventuellement, sa valeur initiale.

PROPRIÉTÉ Syntaxe générale d'un attribut

visibilité nom : type [= valeurParDéfaut]

- **visibilité** : un symbole parmi +, -, #, ~ (voir plus bas) ;
- **nom** : identifiant de l'attribut, en minuscules avec convention CamelCase (nbPages, dateNaissance) ;
- **type** : type de la donnée (int, String, Réel, booléen, Date...) ;
- **= valeurParDéfaut** : optionnel ; valeur que prend l'attribut à la création de l'objet si rien n'est précisé.

EXEMPLE Attributs bien formés

- - solde : Réel = 0.0 : attribut privé solde, de type réel, initialisé à 0 ;
- - nbPages : int : attribut privé entier, sans valeur par défaut ;
- # matricule : String : attribut protégé de type chaîne de caractères ;
- + ville : String = "Yaoundé" : attribut public avec valeur par défaut.

2.5 Syntaxe d'une méthode

Une méthode se décrit par sa signature : son nom, ses paramètres entre parenthèses, et le type de ce qu'elle renvoie.

PROPRIÉTÉ Syntaxe générale d'une méthode

visibilité nom(paramètre : type, ...) : typeRetour

- **nom** : verbe d'action en minuscules (`emprunter`, `consulterSolde`);
- **paramètres** : liste entre parenthèses, chaque paramètre sous la forme `nom : type`, séparés par des virgules ; une méthode sans paramètre garde des parenthèses vides `()` ;
- **typeRetour** : type de la valeur renvoyée — un type primitif (`int`, `Réel`, `booléen`), une classe (`Livre`), ou `void` si la méthode ne renvoie rien.

Direction des paramètres. UML permet de préciser la **direction** d'un paramètre : `in` (le paramètre entre dans la méthode, c'est le cas par défaut) ou `out` (le paramètre sert à faire sortir un résultat). On écrit alors `in montant : Réel`. Au niveau Terminale, on retient simplement que les paramètres ordinaires sont des paramètres `in`.

EXEMPLE Méthodes bien formées

- `+ emprunter() : void` : méthode publique, sans paramètre, ne renvoie rien ;
- `+ estDisponible() : booléen` : méthode publique qui renvoie vrai ou faux ;
- `+ déposer(in montant : Réel) : void` : méthode publique prenant un réel en entrée ;
- `- calculerPenalite(nbJours : int) : Réel` : méthode privée de calcul interne.

2.6 Les quatre niveaux de visibilité

Voici le problème que la visibilité résout. Le solde d'un compte bancaire ne doit jamais être modifié directement par n'importe quel programme : sinon, un bug (ou un fraudeur) pourrait mettre un solde à -5 000 000 FCFA sans passer par un dépôt ou un retrait contrôlé. La **visibilité** est le mécanisme qui protège les données.

DÉFINITION Visibilité

La **visibilité** d'un attribut ou d'une méthode est la règle qui définit qui a le droit d'y accéder depuis l'extérieur de la classe. UML définit quatre niveaux de visibilité, notés par un symbole placé devant l'attribut ou la méthode.

Symbole	Nom	Règle d'accès	Exemple
+	Public	Accessible depuis n'importe quelle classe	+ consulterSolde() : Réel
-	Privé	Accessible uniquement depuis l'intérieur de la classe elle-même	- solde : Réel
#	Protégé	Accessible depuis la classe et ses classes filles (héritage)	# matricule : String
~	Paquetage	Accessible depuis les classes du même paquetage (même dossier logique)	~ codeInterne : int

À RETENIR La règle d'or de l'encapsulation

En général : attributs privés (-), méthodes publiques (+). Les données sont cachées ; on ne les manipule qu'à travers des méthodes contrôlées. C'est le principe d'encapsulation.

LESAVAIS-TU ? Le coffre et le guichet

L'encapsulation fonctionne comme le coffre d'une banque camerounaise : l'argent des clients (les attributs privés) est enfermé dans le coffre ; personne n'y touche directement. Toutes les opérations passent par le guichet (les méthodes publiques) : `deposer()`, `retirer()`, `consulterSolde()`. Le guichetier vérifie chaque opération avant de l'exécuter. De la même façon, la méthode `retirer()` d'une classe `CompteBancaire` vérifie que le solde est suffisant avant de modifier l'attribut privé `solde`. C'est ce principe qui rend les applications comme MTN Mobile Money fiables : ton solde ne peut pas être modifié « à la main », il ne change que via des opérations contrôlées et journalisées.

2.7 Méthode : construire une classe UML pas à pas

MÉTHODE Concevoir la représentation d'une classe

Pour passer d'une description en français à une classe UML correcte :

1. Nommer la classe au singulier, première lettre majuscule, nom en gras dans le compartiment du haut (**Eleve**, pas *eleves*).
2. Lister les attributs dans le compartiment du milieu : pour chacun, choisir un nom significatif et un type précis (`nom : String`, `moyenne : Réel`).
3. Lister les méthodes dans le compartiment du bas : verbe d'action, paramètres typés entre parenthèses, et type de retour systématique (`void` si la méthode ne renvoie rien).

4. Choisir les visibilité : attributs privés (-) et méthodes publiques (+) par défaut ; protégé (#) uniquement si l'attribut doit être accessible aux classes filles.

2.8 Exemple complet : la classe CompteBancaire

EXEMPLE Classe CompteBancaire

Une agence bancaire de Douala veut modéliser ses comptes. Chaque compte possède un numéro et un solde (données sensibles, donc privées), et permet de déposer de l'argent et de consulter le solde (services offerts au client, donc publics).

CompteBancaire
- numero : String - solde : Réel = 0.0
+ deposer(montant : Réel) : void + consulterSolde() : Réel

La classe CompteBancaire : attributs privés, méthodes publiques.

Analysons chaque ligne :

- - solde : Réel = 0.0 : le solde est privé (personne ne le modifie directement) et tout nouveau compte démarre à 0 FCFA ;
- + deposer(montant : Réel) : void : méthode publique qui reçoit un montant en entrée et ne renvoie rien (void) — elle se contente d'augmenter le solde ;
- + consulterSolde() : Réel : méthode publique sans paramètre qui renvoie le solde, un réel.

Pour montrer que cette modélisation n'est pas qu'un dessin, voici comment elle se traduit fidèlement en PHP (langage que tu pratiques en TI) :

```
<?php
class CompteBancaire {
    // Attributs PRIVÉS : inaccessibles depuis l'extérieur
    private string $numero = ""; // initialise pour PHP 8.1+
    private float $solde = 0.0; // valeur par défaut : 0

    // Methode PUBLIQUE : depot d'argent (ne renvoie rien -> void)
    public function deposer(float $montant) : void {
        if ($montant > 0) {
            $this->solde += $montant; // seule voie autorisee
        }
    }
}
```

```
// Methode PUBLIQUE : consultation (renvoie un reel)
public function consulterSolde() : float {
    return $this->solde;
}

$compte = new CompteBancaire();
$compte->deposer(25000); // OK : on passe par le guichet
echo $compte->consulterSolde(); // affiche 25000
// $compte->solde = 999999; // ERREUR : solde est prive !
?>
```

Observe la dernière ligne commentée : tenter de modifier directement `solde` provoque une erreur. C'est exactement ce que le symbole `-` du diagramme annonçait. Le diagramme UML et le code racontent la même histoire.

ATTENTION Erreurs fréquentes

- Oublier le type de retour d'une méthode. Écrire `+ consulterSolde()` est incomplet : il faut `+ consulterSolde() : Réel`. Si la méthode ne renvoie rien, écris explicitement : `void` — ne laisse jamais la signature sans indication.
- Confondre `#` (protégé) et `-` (privé). Le privé interdit tout accès extérieur, y compris aux classes filles ; le protégé autorise l'accès aux classes filles. Au bac, inverser ces deux symboles coûte des points : `#` = famille (classe + filles), `-` = classe seule.
- Nommer la classe au pluriel (`Livres`) ou sans majuscule (`livre`) : la convention impose le singulier avec majuscule initiale.
- Oublier les parenthèses d'une méthode sans paramètre : `emprunter()` et non `emprunter` — sans parenthèses, on croirait lire un attribut.

2.9 Cas particulier : une classe sans méthodes visibles

Il arrive qu'une classe ne serve qu'à stocker des données et n'expose aucune méthode (ou qu'on choisisse de ne pas les montrer à ce stade de la conception). UML l'autorise : le compartiment des méthodes reste alors **vide**, mais le rectangle garde ses trois compartiments — ou, en notation simplifiée, le compartiment vide peut être omis. Par exemple, une classe `Adresse` (`rue : String, ville : String, boitePostale : String`) peut parfaitement apparaître sans méthode dans un premier jet du diagramme. Attention toutefois : seul un compartiment vide est toléré ; un compartiment oublié (attributs absents par négligence) est une erreur de modélisation.

2.10 Exercice résolu

EXERCICE RÉSOLU 2 Modéliser la classe Eleve d'un système de gestion de notes

Énoncé. Le CENADI développe pour un lycée de Bafoussam une application de gestion

des notes. Un élève est caractérisé par son matricule (chaîne), son nom (chaîne) et sa moyenne générale (réel, initialisée à 0). Le système doit permettre de calculer la moyenne à partir d'un nombre de notes et d'une somme de points, et d'afficher les informations de l'élève. La moyenne ne doit jamais être modifiée directement de l'extérieur. Représente la classe `Eleve` en UML.

SOLUTION

Solution détaillée. Appliquons la méthode en quatre étapes.

Étape 1 — Nom. La classe s'appelle `Eleve` (singulier, majuscule).

Étape 2 — Attributs. Trois données : `matricule : String`, `nom : String`, `moyenne : Réel = 0.0` (valeur par défaut imposée par l'énoncé).

Étape 3 — Méthodes. Deux comportements :

- `calculerMoyenne(somme : Réel, nbNotes : int) : void` : elle reçoit deux paramètres en entrée et modifie la moyenne, donc ne renvoie rien ;
- `afficher() : void` : elle affiche, donc ne renvoie rien.

Étape 4 — Visibilités. L'énoncé exige que la moyenne ne soit pas modifiable directement : tous les attributs sont privés (-). Les méthodes sont les services offerts au système : publiques (+).

Eleve
- matricule : String - nom : String - moyenne : Réel = 0.0
+ calculerMoyenne(somme : Réel, nbNotes : int) : void + afficher() : void

Vérification finale : chaque attribut a un type, chaque méthode a parenthèses, paramètres typés et type de retour, et les visibilités respectent l'encapsulation. La classe est correcte.

EXERCICE 1 À toi de jouer

Une startup de Yaoundé développe une application de covoiturage interurbain (Yaoundé-Douala). Un trajet est caractérisé par une ville de départ, une ville d'arrivée, un prix en FCFA (entier) et un nombre de places disponibles (entier, initialisé à 4). Le système doit permettre de réserver une place et de vérifier si le trajet est complet. Représente la classe `Trajet` en UML, en respectant l'encapsulation et toutes les conventions de nommage.

CORRIGÉ Exercice : classe Trajet

Étape 1 : nom Trajet. Étape 2 : attributs privés - villeDepart : String, - villeArrivee : String, - prix : int, - placesDisponibles : int = 4. Étape 3 : méthodes publiques + reserverPlace() : void (décrémente les places, ne renvoie rien) et + estComplet() : booléen (renvoie vrai si placesDisponibles vaut 0). Étape 4 : attributs privés, méthodes publiques — encapsulation respectée. Le rectangle comporte donc le nom en gras, quatre attributs typés (dont un avec valeur par défaut) et deux méthodes avec types de retour explicites.

À RETENIR L'essentiel de la section

- Une **classe** décrit un ensemble d'objets partageant mêmes attributs, mêmes méthodes et même sémantique ; elle se dessine en trois compartiments : nom / attributs / méthodes.
- **Attribut** : visibilité nom : type [= valeurParDéfaut]; **méthode** : visibilité nom(param : type, ...) : typeRetour.
- Quatre visibilitées : + public, - privé, # protégé, ~ paquetage. Règle d'or : attributs privés, méthodes publiques (encapsulation).
- Une méthode qui ne renvoie rien porte le type de retour void ; le nom de la classe est au singulier, en gras, avec majuscule initiale.

Maintenant que tu sais représenter fidèlement une classe — ce moule abstrait —, la question suivante se pose naturellement : comment représente-t-on un objet concret, cette instance particulière avec ses valeurs réelles, comme « le livre ISBN 978-2-07-036-822-4 » ? C'est l'objet de la section suivante.

3 REPRÉSENTATION D'UN OBJET

Dans la section précédente, nous avons appris à représenter une **classe** : un rectangle à trois compartiments (nom, attributs avec leurs types, méthodes). Mais une classe, aussi bien décrite soit-elle, reste une abstraction : la classe `Eleve` décrit ce qu'est un élève en général, pas Amina ni Biya en particulier. Or, quand le proviseur du lycée de Bafoussam consulte les notes du trimestre, il ne manipule pas « l'élève en général » : il manipule cet élève précis, avec ce matricule et ces notes. C'est exactement le rôle de l'**objet** : donner corps, à un instant donné, à ce que la classe décrit. Cette section t'apprend à représenter un objet dans le formalisme UML, à distinguer rigoureusement classe et objet, et à lire un diagramme d'objets.

3.1 De la classe à l'objet : la notion d'instance

Intuition : le moule et les galettes.

Imagine le moule à beignets d'une vendeuse de beignets-haricots à Mokolo. Le moule définit la forme commune de tous les beignets : rond, avec un trou au milieu. Mais chaque beignet vendu est un exemplaire concret : celui-ci est un peu plus doré, celui-là un peu plus gros. Le moule est unique ; les beignets sont nombreux. En UML, la classe joue le rôle du moule, et chaque objet est un beignet : un exemplaire concret, identifiable, avec ses propres caractéristiques.

DÉFINITION Objet (instance d'une classe)

Un **objet** est un **exemplaire concret** d'une classe, appelé aussi **instance**. Il est caractérisé par :

- une **identité** propre, qui le distingue de tous les autres objets, même de ceux qui ont exactement les mêmes valeurs d'attributs ;
- des **valeurs** propres pour chacun des attributs déclarés dans sa classe, à un instant donné ;
- un **comportement** : il peut exécuter les méthodes définies par sa classe.

On dit que l'objet est une instance de sa classe, et que la création d'un objet à partir d'une classe s'appelle l'**instanciation**.

PROPRIÉTÉ Lien d'instanciation

Dans le cadre du programme de Terminale TI, tout objet est l'instance d'une et une seule classe. Cette classe détermine la liste des attributs dont l'objet possède des valeurs et la liste des méthodes qu'il peut exécuter. On dit que l'objet appartient à sa classe, ou qu'il est lié à elle par le lien d'**instanciation**.

Autrement dit : la classe déclare `nom : String, matricule : String` ; l'objet, lui, possède les valeurs `nom = "Amina", matricule = "LB2024-015"`. La classe décrit le type de chaque information ; l'objet porte la valeur de chaque information.

3.2 Notation UML d'un objet

Pourquoi une notation différente de celle de la classe ?

Sur un diagramme, il faut pouvoir distinguer d'un coup d'œil ce qui est un moule (la classe) de ce qui est un exemplaire (l'objet). UML a choisi une convention simple et visuelle : le nom d'un objet est souligné. C'est le signal universel : « attention, ceci est un individu concret, pas une catégorie ».

PROPRIÉTÉ Notation UML d'un objet

Un objet se représente par un **rectangle** (comme une classe), dont le premier compartiment contient le nom de l'objet suivi du nom de sa classe, séparés par deux-points, le

tout **souligné** :

nomObjet : NomClasse

Le deuxième compartiment contient les **valeurs** des attributs, sous la forme `attribut = valeur`, une par ligne. On n'écrit pas les types dans un objet : les types sont déjà déclarés dans la classe. Le compartiment des méthodes est généralement omis pour un objet, car les méthodes sont identiques pour toutes les instances : elles sont définies une fois pour toutes dans la classe.

MÉTHODE Dessiner un objet UML en quatre étapes

Pour représenter correctement un objet :

1. Tracer un rectangle divisé en deux compartiments (le compartiment des méthodes est omis).
2. Dans le premier compartiment, écrire le nom de l'objet en minuscules, suivi de : puis du nom exact de sa classe (avec sa majuscule initiale) : `amina : Eleve`.
3. Souligner l'ensemble `amina : Eleve`. C'est ce soulignement qui fait toute la différence avec une classe.
4. Dans le second compartiment, écrire chaque attribut suivi du signe `=` puis de sa valeur : `nom = "Amina"`. Les chaînes de caractères s'écrivent entre guillemets, les nombres sans guillemets.

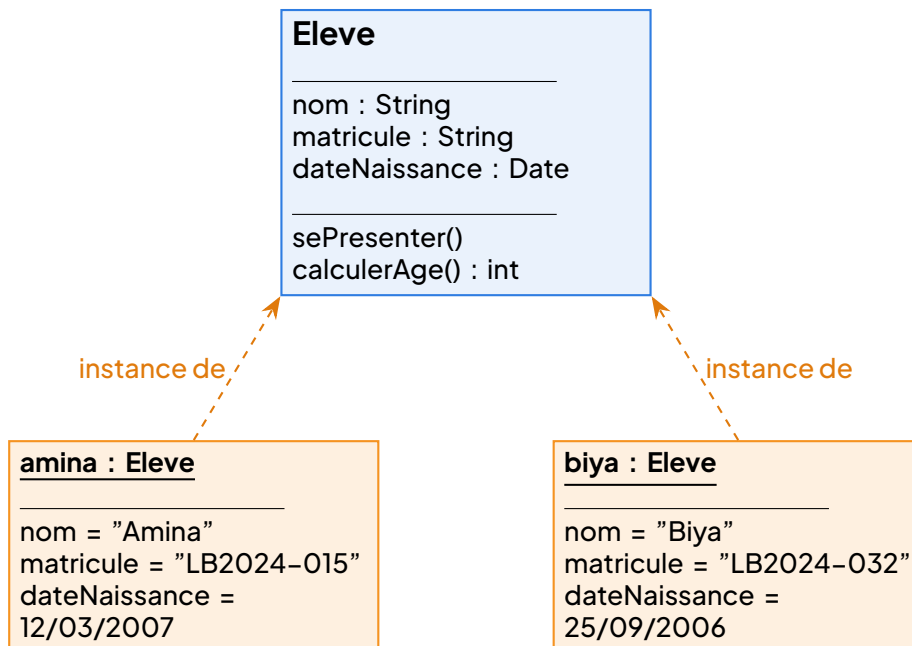
EXEMPLE Deux instances de la classe Eleve

La classe `Eleve` déclare les attributs `nom : String` et `matricule : String`. Voici deux objets instances de cette classe :

`amina : Eleve` avec `nom = "Amina"` et `matricule = "LB2024-015"`

`biya : Eleve` avec `nom = "Biya"` et `matricule = "LB2024-032"`

Ces deux objets ont la même structure (imposée par la classe) mais des valeurs différentes et des identités différentes : même si deux élèves s'appelaient toutes les deux « Amina », ce seraient deux objets distincts, distingués par leur identité (en pratique, souvent par leur matricule).



La classe `Eleve` et deux de ses instances. Les flèches en pointillés matérialisent le lien d'instanciation : chaque objet appartient à une et une seule classe. Remarque : les noms des objets sont soulignés, les valeurs remplacent les types.

3.3 Le compartiment des valeurs

Dans une classe, le deuxième compartiment contient des déclarations : `nbPages : int`. Dans un objet, ce même compartiment contient des constatations : `nbPages = 128`. C'est la différence entre « tout livre possède un nombre de pages qui est un entier » (classe) et « ce livre-ci possède 128 pages » (objet).

PROPRIÉTÉ Syntaxe des valeurs d'attributs

Dans le compartiment des valeurs d'un objet, chaque ligne s'écrit :

`nomAttribut = valeur`

- les chaînes de caractères entre guillemets : `titre = "L'Enfant noir"`;
- les nombres sans guillemets : `nbPages = 128`;
- les dates dans un format lisible : `dateParution = 1953` ou `dateNaissance = 12/03/2007`;
- un attribut dont la valeur n'est pas encore connue peut être noté `attribut = (vide)` ou simplement omis.

ATTENTION Les deux erreurs classiques au baccalauréat

1. Oublier de souligner le nom de l'objet. Sans soulignement, ton rectangle représente une classe nommée « amina : Eleve », ce qui n'a aucun sens : la copie perd des points pour une erreur d'un seul trait de crayon. Souligne systématiquement.
2. Écrire les types au lieu des valeurs : écrire `nom : String` dans un objet est faux. Les types (`String`, `int`...) appartiennent à la classe ; l'objet contient des valeurs concrètes : `nom = "Amina"`. Retiens le réflexe : classe = deux-points et type ; objet = signe égal et valeur.

3.4 Objet nommé et objet anonyme

Parfois, le nom de l'objet n'a pas d'importance : on veut simplement montrer « un exemplaire quelconque de la classe Livre ». UML prévoit alors l'**objet anonyme**.

DÉFINITION Objet anonyme

Un **objet anonyme** est un objet dont on ne précise pas le nom. On le note en omettant la partie avant les deux-points, tout en gardant le soulignement :

 : NomClasse

Par exemple : Livre désigne « un livre quelconque ». On utilise l'objet anonyme quand seule la classe de l'objet (et éventuellement ses valeurs) nous intéresse, pas son identité.

EXEMPLE Objet anonyme

Dans un diagramme d'objets montrant qu'un emprunt concerne « un livre », on peut dessiner un objet anonyme : Livre avec `titre = "L'Enfant noir"`, sans lui donner de nom comme `livre1`. Le soulignement reste obligatoire : un objet anonyme reste un objet.

3.5 Classe ou objet ? La différence à ne jamais confondre

À RETENIR Classe versus objet

	Classe	Objet
Nature	Description abstraite, le « moule »	Exemplaire concret, l'« instance »
Nom	Eleve (non souligné)	<u>amina</u> : Eleve (souligné)
Attributs	Déclarés avec leur type : nom : String	Valorisés : nom = "Amina"
Nombre	Une seule classe Eleve	Autant d'objets que d'élèves réels
Méthodes	Définies dans la classe	Exécutées par l'objet (non ré-écrites)

Le **diagramme d'objets** (ensemble d'objets et de leurs liens à un instant donné) illustre le **diagramme de classes** : il en donne une « photographie » concrète, utile pour vérifier que la modélisation fonctionne sur des cas réels.

EXERCICE RÉSOLU 3 De la classe Livre à ses objets

Énoncé. La bibliothèque du lycée gère son fonds documentaire. La classe `Livre` est définie par les attributs `titre : String`, `auteur : String`, `nbPages : int` et la méthode `estEpais() : bool`. Représenter en notation UML les deux objets suivants : le livre `livre1` de titre « Une vie de boy » de Ferdinand Oyono, 128 pages ; le livre `livre2` de titre « L'Enfant noir » de Camara Laye, 224 pages.

SOLUTION

Solution. Chaque objet est un rectangle à deux compartiments : le nom souligné sous la forme `nom : Classe`, puis les valeurs sous la forme `attribut = valeur`. On n'écrit ni les types, ni la méthode `estEpais()` (définie une fois pour toutes dans la classe).

Premier objet :

livre1 : Livre

titre = "Une vie de boy"

auteur = "Ferdinand Oyono"

nbPages = 128

Second objet :

livre2 : Livre

titre = "L'Enfant noir"

auteur = "Camara Laye"

nbPages = 224

Vérification. Les deux noms sont soulignés ; les chaînes sont entre guillemets ; `nbPages` est un nombre sans guillemets ; aucun type n'apparaît. Chaque objet est lié à la classe `Livre` par le lien d'instanciation, et à une seule classe : le programme est respecté.

EXPÉRIENCE Observer l'instanciation en PHP

Objectif. Vérifier concrètement qu'un objet est une instance de classe, avec ses propres valeurs.

Matériel. Un poste avec WampServer (ou XAMPP) installé, un navigateur, un éditeur de texte.

Manipulation. Crée le fichier `objets.php` dans le dossier `www` et saisis le code suivant :

```
<?php
// La classe : le "moule" (diagramme de classes)
class Eleve {
    public $nom;
    public $matricule;

    public function sePresenter() {
        return "Je suis " . $this->nom .
            ", matricule " . $this->matricule;
    }
}

// Instanciation : creation de deux objets (diagramme d'objets)
$amina = new Eleve(); // objet amina : Eleve
$amina->nom = "Amina"; // nom = "Amina"
$amina->matricule = "LB2024-015";

$biya = new Eleve(); // objet biya : Eleve
$biya->nom = "Biya";
$biya->matricule = "LB2024-032";

echo $amina->sePresenter() . "<br>";
echo $biya->sePresenter() . "<br>";

// Preuve que ce sont deux instances distinctes de la MEME classe
var_dump($amina instanceof Eleve); // bool(true)
var_dump($amina === $biya); // bool(false) : identites differentes
?>
```

Ouvre ensuite `http://localhost/objets.php` dans le navigateur.

Observations. Les deux élèves se présentent avec leurs propres valeurs; `instanceof` confirme que les deux objets appartiennent à la classe `Eleve`; la comparaison stricte montre qu'ils sont distincts.

Interprétation. Chaque appel à `new Eleve()` crée exactement ce qu'un diagramme d'objets représente : une nouvelle instance, dotée d'une identité propre et de valeurs propres, rattachée à une et une seule classe.

LESAVAIS-TU ? Du diagramme d'objets à la mémoire de l'ordinateur

En programmation orientée objet, l'instruction `new Eleve()` (en PHP, Java ou Python) demande à l'ordinateur de réserver une zone de mémoire pour un nouvel objet : c'est précisément ce que tu dessines quand tu traces un rectangle souligné `amina : Eleve`. Les dé-

veloppeurs utilisent d'ailleurs les diagrammes d'objets pour raisonner sur l'état de la mémoire à un instant donné, par exemple pour traquer un bug dans une application Mobile Money : « à 14 h 32, l'objet transaction : Paiement avait `montant = 5000` et `statut = "en attente"` ». Le diagramme d'objets est la photographie ; le diagramme de classes est le plan de construction.

EXERCICE 2 À toi de jouer

On considère la classe `Compte` définie par les attributs `numero : String`, `solde : float` et la méthode `deposer(montant : float)`.

1. Représente en notation UML l'objet `c1 : Compte` avec `numero = "0M-699123456"` et `solde = 75000`.
2. Représente un objet anonyme de la classe `Compte` dont le solde est nul.
3. Un camarade a écrit dans son objet `solde : float`. Releve l'erreur et corrige-la.

CORRIGÉ Exercice 1

1. Rectangle à deux compartiments. Premier compartiment : `c1 : Compte` (souligné). Second compartiment : `numero = "0M-699123456"` et `solde = 75000`. La méthode `deposer` n'apparaît pas : elle est définie dans la classe.
2. Rectangle dont le premier compartiment contient `: Compte` (souligné, sans nom avant les deux-points) et le second `solde = 0`.
3. L'erreur est d'écrire le type (`solde : float`) dans un objet : les types appartiennent à la classe. Dans un objet, on écrit une valeur : `solde = 75000` par exemple. On vérifiera aussi que le nom de l'objet est bien souligné.

À RETENIR L'essentiel de la section

- Un **objet** est une **instance** concrète d'une classe : il possède une identité et des valeurs propres à un instant donné.
- Notation UML : rectangle dont le nom `nomObjet : NomClasse` est **souligné** ; objet anonyme : `: NomClasse` souligné.
- Compartiment des valeurs : `attribut = valeur` (jamais les types, jamais les méthodes).
- Tout objet appartient à une et une seule classe (lien d'instanciation, flèche en poin-

tillés).

- Le diagramme d'objets est la photographie concrète qui illustre et valide le diagramme de classes.

4

ASSOCIATIONS ET MULTIPLICITÉS

Dans la section précédente, tu as appris à représenter un **objet** : une instance concrète d'une classe, avec ses valeurs d'attributs propres (par exemple l'objet `e1` : `Eleve` dont l'attribut `nom` vaut « Kamga »). Mais un système d'information réel ne contient pas des objets isolés qui flottent chacun dans leur coin. Dans le logiciel de gestion de la bibliothèque du lycée, un élève emprunte des livres ; dans l'application de gestion des notes, un professeur enseigne des cours ; dans une application Mobile Money, un client effectue des transactions. Ces liens entre objets sont la matière même du système. Comment les modéliser proprement, avant d'écrire la moindre ligne de code ou la moindre table SQL ? C'est exactement le rôle des **associations** et de leurs **multiplicités**, que nous étudions maintenant.

4.1 La notion d'association

4.2 Pourquoi relier les classes ?

Imagine que tu développes le système d'information de la bibliothèque de ton lycée. Tu as déjà identifié deux classes : `Eleve` (attributs : `matricule`, `nom`, `classe`) et `Livre` (attributs : `isbn`, `titre`, `auteur`). Chacune est bien définie. Pourtant, une question essentielle reste sans réponse : qui a emprunté quoi ? Ni la classe `Eleve` seule, ni la classe `Livre` seule ne peuvent y répondre. Il manque une information structurelle : le fait que des objets élèves et des objets livres sont **liés** entre eux de façon durable (et non pas seulement le temps d'un calcul). C'est ce lien que l'on modélise par une association.

DÉFINITION Association

Une **association** est une relation structurelle entre deux classes (ou plus) qui indique que des objets de ces classes sont liés entre eux de manière durable dans le système. Dans le formalisme UML, une association se matérialise par un trait plein reliant les classes concernées. Chaque objet concret relié à un autre par cette association est appelé une instance de l'association (un lien).

ATTENTION Association ou simple utilisation ?

Ne confonds pas une association avec une simple utilisation temporaire. Si une méthode

de `Eleve` reçoit un objet `Livre` en paramètre juste pour un calcul ponctuel, ce n'est pas forcément une association. Une association traduit un lien mémorisé par le système : la bibliothèque doit se souvenir que Kamga a emprunté tel livre, tant que l'emprunt n'est pas clôturé. C'est cette persistance du lien qui justifie le trait plein.

4.3 Nom de l'association et sens de lecture

Une association porte généralement un **nom**, placé au milieu du trait. Par convention, on utilise un **verbe** (ou un groupe verbal) qui exprime l'action reliant les objets : emprunter, enseigner, passer (une commande), travailler dans (un service).

Comme un verbe a un sens (« l'élève emprunte le livre », et non l'inverse !), UML permet d'ajouter un petit **triangle de direction** (auss appelé flèche de sens de lecture) accolé au nom. Il indique dans quel sens on lit la phrase :

Eleve ► emprunter se lit : « un élève emprunte un livre »

PROPRIÉTÉ Règles de nommage d'une association

- Le nom d'une association est un verbe à l'infinitif ou un groupe verbal, écrit en minuscules (convention usuelle) : emprunter, enseigner.
- Le triangle de direction ► ou ◄ précise le sens de lecture de la phrase ; il ne s'agit pas d'une flèche de navigation ni d'un sens de parcours obligatoire dans le code.
- Si le nom est évident, on peut l'omettre, mais c'est déconseillé au baccalauréat : un nom explicite rend le diagramme auto-documenté.

4.4 Les rôles aux extrémités

En plus du nom de l'association, on peut préciser, à **chaque extrémité** du trait, le **rôle** joué par la classe dans le lien. Le rôle est un nom commun qui répond à la question : « dans ce lien, l'objet joue le rôle de quoi ? »

Par exemple, dans l'association travailler dans entre `Employe` et `Entreprise`, on peut indiquer le rôle salarié du côté de `Employe` et le rôle employeur du côté de `Entreprise`. Dans notre bibliothèque, `Eleve` joue le rôle d'emprunteur et `Livre` celui d'ouvrage emprunté.

À RETENIR Nom, sens de lecture, rôles

Une association se décrit par trois éléments complémentaires et facultatifs :

1. le nom (un verbe) au milieu du trait ;
2. le triangle de direction indiquant le sens de lecture ;

3. les rôles, placés près de chaque extrémité, nommant la fonction de chaque classe dans le lien.

4.5 La multiplicité (cardinalité)

4.6 Le problème : combien d'objets de chaque côté ?

Reprenons la bibliothèque. Dire qu'un élève emprunte des livres ne suffit pas au concepteur : faut-il prévoir en mémoire (et plus tard en base de données) la place pour un seul livre par élève, ou pour plusieurs ? Un livre peut-il être emprunté par plusieurs élèves en même temps ? Un élève qui vient de s'inscrire et n'a encore rien emprunté est-il un cas légal ? Toutes ces réponses conditionnent la structure du programme et de la base de données. La **multiplicité** est l'outil UML qui les exprime de façon concise et rigoureuse.

DÉFINITION Multiplicité

La **multiplicité** (ou cardinalité) est une indication placée à chaque extrémité d'une association, qui précise le nombre minimal et maximal d'objets de la classe située à cette extrémité pouvant être liés à un objet de la classe d'en face.

PROPRIÉTÉ Notations de multiplicité

Notation	Signification
1	exactement un (ni zéro, ni plusieurs)
0..1	au plus un (zéro ou un)
1..*	au moins un (un ou plusieurs, sans limite)
*	zéro ou plusieurs (aucune contrainte)
n..m	entre n et m objets (intervalle borné, ex. 2..5)

Le symbole * se lit « plusieurs » et signifie « un nombre quelconque, non borné ».

4.7 La lecture croisée : la règle d'or

Voici le point le plus délicat — et le plus souvent sanctionné au baccalauréat. La multiplicité placée à une extrémité ne décrit **pas** la classe à laquelle elle est accolée : elle décrit **la classe d'en face**, vue depuis un objet de la classe de départ.

MÉTHODE Lire correctement une association et ses multiplicités

1. Choisis un sens de lecture, par exemple de A vers B.
2. Pose la question : « un objet A est associé à combien d'objets B ? »
3. La réponse est la multiplicité écrite à l'extrémité de B, c'est-à-dire en face de ton point de départ.
4. Recommence dans l'autre sens : « un objet B est associé à combien d'objets A ? » — réponse : la multiplicité du côté de A.
5. Formule les deux phrases en français courant, avec le nom de l'association.

ATTENTION Erreur classique : la multiplicité du mauvais côté

L'erreur la plus fréquente consiste à placer (ou à lire) la multiplicité du côté de la classe de départ. Retiens bien : la multiplicité se lit « en face », à l'extrémité opposée de la classe dont tu pars. Si `Eleve 1 — 0..* Livre`, c'est le `0..*` (côté `Livre`) qui dit combien de livres un élève emprunte, et le `1` (côté `Eleve`) qui dit par combien d'élèves un livre est emprunté. Astuce mnémotechnique : « je regarde en face pour savoir combien j'en ai ».

4.8 Exemple guidé : l'emprunt à la bibliothèque

Appliquons la méthode à notre système de bibliothèque. La règle de gestion de l'établissement est la suivante : « un élève peut emprunter plusieurs livres (ou aucun à un instant donné); un livre ne peut être emprunté que par un seul élève à la fois ».



Association emprunter entre Eleve et Livre, avec nom, triangle de sens de lecture, rôles et multiplicités.

Lecture croisée, étape par étape :

Sens de lecture	Phrase obtenue
De Eleve vers Livre (multiplicité <code>0..*</code> lue en face, côté Livre)	Un élève emprunte zéro ou plusieurs livres.
De Livre vers Eleve (multiplicité <code>1</code> lue en face, côté Eleve)	Un livre est emprunté par exactement un élève (à la fois).

Vérifie la cohérence avec la règle de gestion : « un élève peut n'avoir rien emprunté » est bien couvert par le 0 de 0..*; « un livre n'est pas partagé entre deux élèves simultanément » est garanti par le 1 strict. Le modèle et la réalité concordent : le diagramme est correct.

4.9 Un second exemple chiffré : professeurs et cours

EXEMPLE Association enseigner entre Professeur et Cours

Règles de gestion du lycée : « tout professeur enseigne au moins un cours ; un cours peut n'être encore attribué à aucun professeur (par exemple une nouvelle option créée en septembre), ou être assuré par plusieurs professeurs (cours mutualisé entre deux classes) ».



Lecture croisée :

- Un professeur enseigne au moins un cours (1..* côté Cours) : impossible d'avoir un professeur sans aucun cours dans le système.
- Un cours est enseigné par zéro ou plusieurs professeurs (0..* côté Professeur) : le 0 autorise le cours non encore attribué.

LESAVAIS-TU ? Des multiplicités UML aux cardinalités du modèle entité-association

En classe de Première, tu as étudié le modèle entité-association (modèle de Merise), où les liens entre entités portent des cardinalités notées 1,1, 0,1, 1,n, 0,n. Bonne nouvelle : les multiplicités UML correspondent exactement à ce concept ! La correspondance est directe : 0..1 $\leftrightarrow$ 0,1 ; 1 $\leftrightarrow$ 1,1 ; 0..* $\leftrightarrow$ 0,n ; 1..* $\leftrightarrow$ 1,n. UML et Merise sont deux langages différents pour exprimer la même réalité structurale. C'est d'ailleurs grâce à ces multiplicités qu'on déterminera plus tard, lors du passage à la base de données relationnelle, où placer les clés étrangères : une multiplicité 1 en face impose généralement une clé étrangère côté « plusieurs ».

4.10 La classe-association : quand le lien porte des informations

4.11 Le problème : où mettre la date d'emprunt ?

Reprenons notre bibliothèque et posons une question en apparence simple : où ranger l'attribut *dateEmprunt* ?

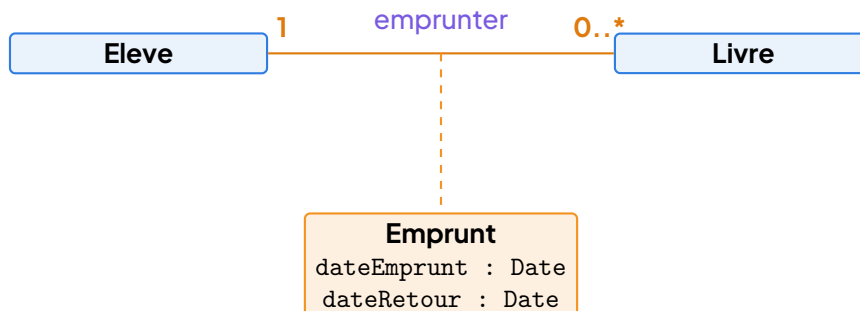
- Dans *Eleve* ? Non : un élève qui emprunte trois livres a trois dates d'emprunt différentes, une par livre.

- Dans `Livre`? Non plus : au fil du temps, un même livre est emprunté à plusieurs reprises, avec une date différente à chaque fois.

La date d'emprunt n'appartient ni à l'élève ni au livre : elle appartient **au lien lui-même**, c'est-à-dire à chaque couple (élève, livre). UML résout élégamment ce problème avec la **classe-association**.

DÉFINITION Classe-association

Une **classe-association** est une classe rattachée à une association par un trait en pointillés. Elle permet de doter le lien lui-même d'attributs (et éventuellement de méthodes) qui ne dépendent d'aucune des deux classes prises isolément, mais de leur combinaison. Chaque instance du lien (chaque couple d'objets reliés) possède ses propres valeurs pour ces attributs.



Classe-association `Emprunt` rattachée par un trait pointillé au lien `Eleve—Livre` : les dates appartiennent à chaque emprunt, pas à l'élève ni au livre.

EXEMPLE Lecture de la classe-association `Emprunt`

Avec le schéma ci-dessus, chaque emprunt concret possède ses propres dates :

- Kamga emprunte « Les Soleils des indépendances » : `dateEmprunt` = 12/09/2024, `dateRetour` = 26/09/2024.
- Kamga emprunte « Une si longue lettre » : `dateEmprunt` = 03/10/2024, `dateRetour` = (non renseignée, emprunt en cours).

Les deux emprunts concernent le même élève, mais ce sont deux liens distincts, donc deux instances de `Emprunt` avec des valeurs différentes. Plus tard, en base de données, cette classe-association deviendra naturellement une table `Emprunt` contenant les clés étrangères de l'élève et du livre, plus les colonnes de dates.

ATTENTION Quand utiliser (et ne pas utiliser) une classe-association

N'utilise une classe-association que si le lien porte réellement des informations propres (date, quantité, note, prix au moment de la commande...). Si le lien n'a aucun attribut, un simple trait d'association suffit : ajouter une classe-association vide alourdit le diagramme sans rien apporter. Autre piège : ne duplique pas dans `Eleve` ou `Livre` un attribut déjà placé dans la classe-association — la redondance est une faute de conception.

4.12 Du diagramme de classes au diagramme d'objets : le lien

Nous avons vu à la section précédente qu'un **objet** est une instance d'une classe. De la même façon, un **lien** est une **instance d'une association** : c'est la connexion concrète entre deux objets précis. Dans un diagramme d'objets, le lien se dessine par un **trait plein** entre les objets, exactement comme l'association relie les classes dans le diagramme de classes.

EXEMPLE Lien entre objets : l'emprunt de Kamga

Diagramme d'objets correspondant à l'association emprunter :



Ce schéma dit : l'objet `e1` (Kamga) est lié à l'objet `l1` (le livre de Mariama Bâ) par une instance de l'association `emprunter`. Si Kamga emprunte un second livre `l2`, on tracerait un second lien de `e1` vers `l2` — ce qui est cohérent avec la multiplicité `0..*` côté `Livre`. En revanche, dessiner un lien de `l1` vers un autre élève `e2` en même temps violerait la multiplicité `1` côté `Eleve` : le diagramme d'objets est donc un excellent moyen de vérifier que les multiplicités choisies reflètent bien la réalité.

À RETENIR Classe/objet, association/lien : le parallèle fondamental

- Une classe décrit des objets ; un objet est une instance de classe.
- Une association décrit des liens ; un lien est une instance d'association.
- Le diagramme de classes montre les associations et leurs multiplicités ; le diagramme d'objets montre les liens concrets, et doit respecter ces multiplicités.

4.13 Exercice résolu et entraînement

EXERCICE RÉSOLU 4**Modéliser les commandes d'une boutique en ligne**

Énoncé. Une startup doualaïse de vente en ligne veut modéliser son système. Règles de gestion : « un client peut passer plusieurs commandes, ou aucune s'il vient de créer son compte ; toute commande est passée par exactement un client ; chaque commande concerne au moins un produit, et un produit peut figurer dans plusieurs commandes ou dans aucune ». On note aussi que chaque fois qu'un produit figure dans une commande, on mémorise la `quantite` commandée et le `prixUnitaire` pratiqué à ce moment-là.

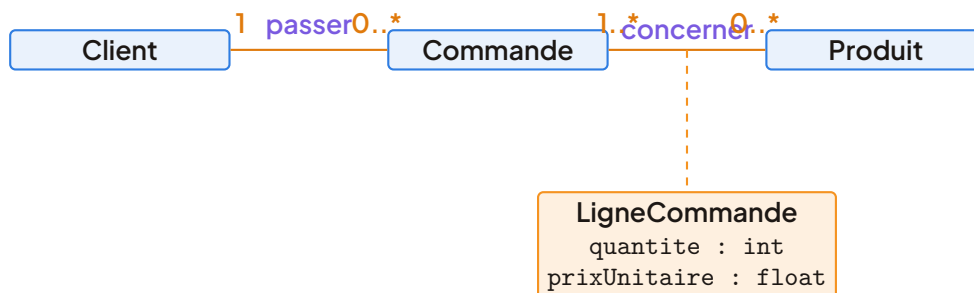
1. Dessine le diagramme de classes avec les associations et multiplicités.
2. Où places-tu `quantite` et `prixUnitaire` ? Justifie.

SOLUTION

Solution détaillée.

1. On identifie trois classes : `Client`, `Commande`, `Produit`, et deux associations :

- passer entre `Client` et `Commande` : un client passe 0..* commandes (multiplicité côté `Commande`) ; une commande est passée par 1 client (multiplicité côté `Client`).
- concerner entre `Commande` et `Produit` : une commande concerne 1..* produits (côté `Produit`) ; un produit figure dans 0..* commandes (côté `Commande`).



2. `quantite` et `prixUnitaire` ne peuvent aller ni dans `Commande` (une commande contient plusieurs produits, donc plusieurs quantités) ni dans `Produit` (un même produit est commandé en quantités différentes selon les commandes, et son prix catalogue peut évoluer alors que le prix au moment de la commande doit rester archivé). Ils appartiennent au lien entre une commande et un produit : on crée donc la classe-association `LigneCommande`, rattachée en pointillés à l'association `concerner`.

EXERCICE 3**À toi de jouer : le cybercafé de quartier**

Un cybercafé de Yaoundé veut gérer ses sessions. Règles de gestion : « un client peut utiliser plusieurs postes au fil du temps (mais un seul à la fois n'est pas exigé par le modèle) ; un poste peut être utilisé par plusieurs clients successivement, ou par aucun s'il est neuf ;

pour chaque session, on mémorise l'heure de début et l'heure de fin ».

1. Dessine l'association entre `Client` et `Poste` avec les multiplicités adéquates.
2. Justifie l'emplacement des attributs `heureDebut` et `heureFin`, puis complète le diagramme.
3. Dessine un diagramme d'objets montrant deux clients et trois postes, avec des liens respectant tes multiplicités.

CORRIGÉ Exercice : le cybercafé de quartier

1. Association utiliser entre `Client` et `Poste` : un client utilise 0..* postes (au fil du temps ; le 0 couvre le client inscrit qui n'a encore rien utilisé) ; un poste est utilisé par 0..* clients (successivement ; le 0 couvre le poste neuf). On a donc `Client 0..* — 0..* Poste` : c'est une association plusieurs-à-plusieurs.
2. `heureDebut` et `heureFin` ne dépendent ni du client seul (il a plusieurs sessions) ni du poste seul (il a servi à plusieurs sessions) : elles dépendent du couple (client, poste, moment). On crée une classe-association `Session` rattachée en pointillés au lien utiliser, portant `heureDebut : Heure` et `heureFin : Heure`.
3. Diagramme d'objets : par exemple `c1 : Client` lié à `p1 : Poste` et à `p2 : Poste` (deux sessions différentes), `c2 : Client` lié à `p1` (un autre jour) ; `p3 : Poste` n'est lié à personne (poste neuf). Aucune multiplicité n'est violée : chaque extrémité accepte 0..*.

Tu maîtrises désormais les associations, leurs noms, leurs rôles, leurs multiplicités — et la règle d'or de la lecture croisée « en face ». Il reste un dernier chantier structurel : exprimer des liens plus forts que la simple association, à savoir l'héritage, l'agrégation et la composition. C'est l'objet de la section suivante.

5 HÉRITAGE, AGRÉGATION ET COMPOSITION

Dans la section précédente, tu as appris à relier des classes par des **associations** et à préciser le nombre d'instances concernées grâce aux **multiplicités**. Mais toutes les relations entre classes ne sont pas de simples associations. Reprends l'exemple du système d'information de ton lycée : un `Eleve` et un `Enseignant` ont beaucoup en commun (un nom, une date de naissance, un téléphone...). Faut-il recopier ces attributs dans les deux classes ? Et quand une classe est « faite de » plusieurs autres (un bulletin est fait de lignes de notes), s'agit-il d'une association ordinaire ou de quelque chose de plus fort ? C'est précisément pour répondre à ces deux questions qu'UML propose trois relations spéciales : l'héritage, l'agrégation et la composition.

5.1 La relation d'héritage (généralisation / spécialisation)

Intuition : le problème de la duplication.

Imagine que tu modélises le lycée. Tu écris une classe `Eleve` avec les attributs `nom`, `dateNaissance`, `matricule`, puis une classe `Enseignant` avec `nom`, `dateNaissance`, `matiere`. Les attributs `nom` et `dateNaissance` sont écrits deux fois. Conséquences : si demain on ajoute l'attribut `telephone`, il faudra penser à l'ajouter partout ; si on corrige une méthode `calculerAge()`, il faudra la corriger partout. C'est une source certaine d'incohérences et de bugs. L'héritage résout ce problème : on factorise ce qui est commun dans une classe mère, et chaque classe fille ne contient que ce qui lui est propre.

DÉFINITION Héritage

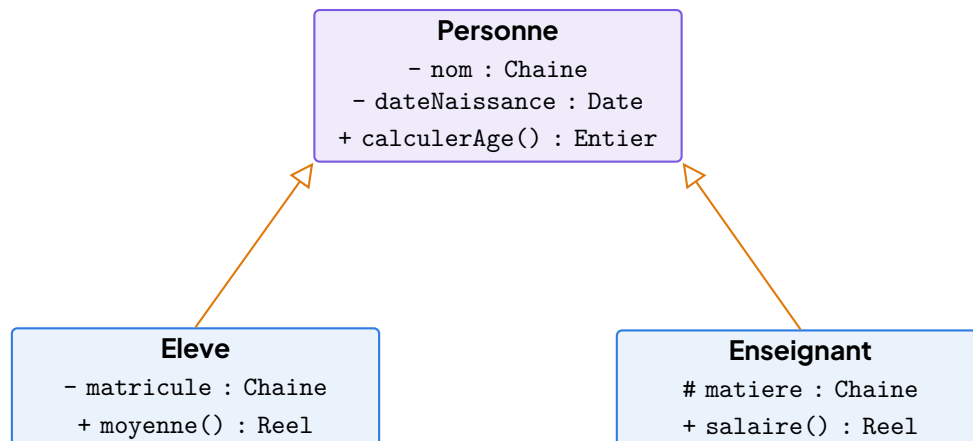
L'**héritage** (ou généralisation/spécialisation) est une relation par laquelle une classe, dite **classe fille** (ou sous-classe, ou classe spécialisée), récupère automatiquement les attributs et les méthodes d'une autre classe, dite **classe mère** (ou super-classe, ou classe générale), et peut en ajouter de nouveaux. On dit que la classe fille spécialise la classe mère, et que la classe mère généralise les classes filles.

PROPRIÉTÉ Notation UML de l'héritage

L'héritage se représente par une flèche à triangle vide (trait plein, tête triangulaire non remplie) qui part de la classe fille et pointe vers la classe mère. Les attributs et méthodes hérités ne se recopient pas dans les classes filles : ils sont présents implicitement. On n'écrit dans une classe fille que ce qu'elle ajoute (ou redéfinit).

EXEMPLE Hiérarchie Personne – Eleve – Enseignant

La classe mère `Personne` porte `nom` et `dateNaissance`. La classe fille `Eleve` ajoute `matricule` ; la classe fille `Enseignant` ajoute `matiere`. Un objet `Eleve` possède donc, au total, trois attributs : `nom`, `dateNaissance` (hérités) et `matricule` (propre).



Hiérarchie d'héritage : les flèches à triangle vide partent des classes filles (Eleve, Enseignant) et pointent vers la classe mère (Personne). Les attributs `nom` et `dateNaissance` ne sont pas recopiés dans les classes filles.

La visibilité protégée (#).

Tu connais déjà les visibilités `+` (publique : accessible partout) et `-` (privée : accessible uniquement dans la classe elle-même). L'héritage introduit un besoin nouveau : comment permettre à `Enseignant` de manipuler directement un attribut de `Personne` sans l'ouvrir à tout le monde ? Réponse : la visibilité **protégée**, notée `#`.

DÉFINITION Visibilité protégée

Un membre (attribut ou méthode) déclaré **protégé** (`#`) est accessible dans la classe où il est défini et dans toutes ses classes descendantes (filles, petites-filles...), mais inaccessible depuis l'extérieur de la hiérarchie.

EXEMPLE Lecture de la hiérarchie

Dans la figure ci-dessus, si `dateNaissance` était déclarée `# dateNaissance : Date`, alors la méthode `salaire()` de `Enseignant` pourrait la lire directement (par exemple pour calculer une prime d'ancienneté), tandis qu'un programme extérieur ne pourrait y accéder que via une méthode publique comme `calculerAge()`.

ATTENTION Piège n°1 : le sens de la flèche d'héritage

L'erreur la plus fréquente au Baccalauréat est d'orienter la flèche vers la classe fille. Retiens cette phrase : « la flèche montre où l'on puise ». La classe fille puise ses caractéris-

tiques dans la classe mère : la flèche part donc de la fille et le triangle vide touche la mère. Autre moyen mnémotechnique : le triangle pointe vers le « parent », comme un enfant qui lève les yeux vers son père ou sa mère. Une flèche `Personne` → `Eleve` est fausse.

5.2 L'agrégation : une relation tout/partie faible

Intuition.

Certaines classes sont naturellement « composées de » d'autres : une classe (scolaire) regroupe des élèves, une équipe de football regroupe des joueurs, une playlist regroupe des chansons. Mais attention : si l'on supprime la classe « Terminale TI » du système d'information du lycée, les élèves ne disparaissent pas pour autant — ils existent toujours, ils seront réaffectés ailleurs. La « partie » a une vie propre, indépendante du « tout ». C'est exactement ce que modélise l'agrégation.

DÉFINITION Agrégation

L'**agrégation** est une association particulière de type tout/partie (le « tout » est appelé **contenant** ou agrégat, la « partie » est le **contenu**) dans laquelle la partie peut exister indépendamment du tout. Elle se représente par un trait reliant les deux classes, avec un losange vide (◊) placé du côté du contenant.

EXEMPLE Classe ◊ — Eleve

Une `Classe` agrège des `Eleve` : multiplicité 1 côté `Classe` (avec le losange vide) et 0..* côté `Eleve`. Un élève peut n'appartenir à aucune classe (un nouvel inscrit pas encore affecté), et il survit à la suppression de sa classe.

5.3 La composition : une relation tout/partie forte

Intuition.

Prenons maintenant un `Bulletin` et ses `LigneNote` (une ligne par matière : note, coefficient, appréciation). Si l'on supprime le bulletin, les lignes de notes n'ont plus aucune raison d'exister : elles disparaissent avec lui. De même, une `Page` n'existe pas sans son `Document`, une `Piece` n'existe pas sans sa `Maison`. La durée de vie de la partie est liée à celle du tout : c'est la composition.

DÉFINITION Composition

La **composition** est une agrégation forte dans laquelle la durée de vie de la partie dépend du tout : la partie est créée avec le tout, appartient à un seul tout à la fois, et est détruite

avec lui. Elle se représente par un trait reliant les deux classes, avec un losange plein (◆) placé du côté du contenant.

PROPRIÉTÉ Règle UML sur la composition (admise)

Dans une composition, la multiplicité côté contenant (côté du losange plein) est au plus 1 : elle vaut 1 ou 0..1, jamais *. En effet, une partie ne peut pas « appartenir fortement » à plusieurs tous simultanément. Cette règle est admise au programme de Terminale TI : signale-la simplement lorsque tu dessines une composition.



Agrégation : l'élève **survit** à la suppression de la classe



Composition : la page **ne survit pas** à la suppression du document

Comparatif : en haut, agrégation (losange vide côté contenant) — la partie survit ; en bas, composition (losange plein côté contenant) — la partie ne survit pas. Les multiplicités s'écrivent comme pour une association ordinaire.

5.4 Comment choisir entre agrégation et composition ?

MÉTHODE Le test de la dépendance d'existence

Face à une relation « tout/partie », pose-toi une seule question, dans cet ordre :

1. Si je supprime le tout, la partie a-t-elle encore un sens, une existence propre ?
2. Si oui (la partie survit, peut être réutilisée ou partagée) : c'est une agrégation → losange vide côté contenant.
3. Si non (la partie disparaît avec le tout, n'appartient qu'à lui) : c'est une composition → losange plein côté contenant.
4. Place ensuite les multiplicités des deux côtés, comme pour une association. Vérifie

enfin la règle : en composition, côté losange plein, multiplicité ≤ 1 .

EXEMPLE Exemple chiffré : le critère en action

- Le lycée supprime la classe « Terminale TI » de son logiciel de gestion : ses 52 élèves existent toujours dans la base (ils seront réaffectés ou archivés). La partie survit $\Rightarrow$ agrégation : `Classe` $\diamond$ — `Eleve`.
- On supprime un Bulletin : ses 14 `LigneNote` (une par matière) disparaissent avec lui, car une ligne de note sans bulletin ne signifie rien. La partie ne survit pas $\Rightarrow$ composition : `Bulletin` $\blacklozenge$ — `LigneNote`, avec 1 côté bulletin et 1..* côté lignes.

ATTENTION Piège n°2 : placer le losange du mauvais côté

Deuxième erreur classique : dessiner le losange (vide ou plein) côté partie au lieu du côté contenant. Retiens : « le losange appartient au tout » — c'est le tout qui « porte » le losange, comme une boîte porte sa serrure. Dans `Classe` $\diamond$ — `Eleve`, le losange touche `Classe`, jamais `Eleve`. Vérifie aussi que tu n'utilises pas un losange plein quand la partie peut être partagée (un même élève pourrait figurer dans un club et une équipe : jamais de composition dans ce cas).

EXERCICE RÉSOLU 5 Modéliser héritage, agrégation et composition dans un même système

Énoncé. Le CENADI veut modéliser le personnel d'une administration. Un `Agent` possède un `nom` et un `numeroCNI`. Un `Informaticien` est un agent qui possède en plus une `specialite`; un `Comptable` est un agent qui possède en plus un `numeroOrdre`. Un `Service` regroupe plusieurs agents, qui survivent à la fermeture du service. Un `Dossier` est constitué de `PieceJointe` qui disparaissent avec lui. Représente ces relations en précisant les notations UML utilisées.

SOLUTION

Solution détaillée.

1. Héritage : `Informaticien` et `Comptable` partagent `nom` et `numeroCNI` $\Rightarrow$ classe mère `Agent` portant ces deux attributs ; flèches à triangle vide depuis `Informaticien` (qui n'écrit que `specialite`) et `Comptable` (qui n'écrit que `numeroOrdre`) vers `Agent`.
2. Agrégation : les agents survivent à la fermeture du service (dépendance d'existence négative) $\Rightarrow$ losange vide côté `Service`, multiplicités 1 côté `Service` et 0..* côté `Agent`.

3. Composition : les pièces jointes disparaissent avec le dossier $\Rightarrow$ losange plein côté Dossier, multiplicités 1 côté Dossier (règle : au plus 1 côté contenant) et 1..* côté PieceJointe.

Le diagramme final comporte donc six classes, deux flèches d'héritage correctement orientées vers la mère, une agrégation et une composition, chacune avec ses multiplicités.

EXERCICE 4 À toi de jouer

Modélise la bibliothèque municipale de Bafoussam : un Adherent et un Bibliothecaire sont tous deux des Personne (nom, telephone) ; l'adhérent a un numeroCarte, le bibliothécaire un grade. Une Bibliotheque regroupe des Livre qui continuent d'exister si la bibliothèque ferme (ils sont transférés). Un Catalogue est formé de Fiche qui n'ont aucune existence sans lui. Indique pour chaque relation sa nature (héritage, agrégation, composition), le sens de la notation et les multiplicités.

CORRIGÉ Exercice : bibliothèque de Bafoussam

- Héritage : Personne (nom, telephone) est la classe mère ; flèches à triangle vide depuis Adherent (numeroCarte) et Bibliothecaire (grade) vers Personne. Attributs hérités non recopiés.
- Agrégation : Bibliotheque $\diamond$ —Livre (losange vide côté Bibliotheque) car les livres survivent ; multiplicités 1 / 0..*.
- Composition : Catalogue $\blacklozenge$ —Fiche (losange plein côté Catalogue) car les fiches disparaissent avec le catalogue ; multiplicités 1 / 1..*, le côté contenant restant bien ≤ 1 .

À RETENIR L'essentiel de la section

- Héritage : la classe fille récupère attributs et méthodes de la classe mère ; flèche à triangle vide de la fille vers la mère ; attributs hérités jamais recopiés ; visibilité # = accessible dans la classe et ses descendantes.
- Agrégation : tout/partie faible, la partie survit au tout ; losange vide côté contenant.
- Composition : tout/partie forte, la partie meurt avec le tout ; losange plein côté contenant ; multiplicité côté contenant au plus 1.
- Critère de choix : la partie a-t-elle une existence propre ? Oui $\Rightarrow$ agrégation ; non $\Rightarrow$ composition.

- Les multiplicités s'écrivent sur les agrégations et compositions comme sur les associations.

LESAVAIS-TU ? L'héritage dans les vraies applications camerounaises

Les grandes plateformes de paiement mobile utilisées au Cameroun s'appuient sur des hiérarchies de classes : une classe mère *Transaction* (montant, date, référence) est spécialisée en *Depot*, *Retrait* et *Transfert*, chacune ajoutant ses propres règles de calcul de frais. Factoriser le code commun dans la classe mère, c'est moins de bugs, des mises à jour plus rapides... et des millions de transactions sécurisées chaque jour.

6 PRODUIRE UN DIAGRAMME DE CLASSES ET UN DIAGRAMME D'OBJETS

Dans la section précédente, tu as appris à distinguer l'**héritage**, l'**agrégation** et la **composition**, et à dessiner chacun de ces liens avec le bon symbole UML. Tu possèdes donc désormais toutes les « pièces détachées » du langage : classes, objets, attributs, méthodes, associations, multiplicités. Il reste à assembler ces pièces pour construire un modèle complet à partir d'un énoncé en français. C'est exactement ce que l'on te demandera au Baccalauréat TI : on te donne une situation réelle (une bibliothèque, un cybercafé, une école) et tu dois produire un **diagramme de classes** cohérent, puis un **diagramme d'objets** qui l'illustre à un instant donné. Cette section te donne une démarche en cinq étapes, une étude de cas complète guidée pas à pas, et les critères de vérification qui feront la différence le jour de l'examen.

6.1 De l'énoncé au diagramme : la démarche en cinq étapes

Face à un texte descriptif, le piège est de se lancer directement dans le dessin. Le bon concepteur procède méthodiquement, comme un enquêteur qui relève des indices. L'idée directrice est simple : **les noms désignent des classes candidates, les verbes désignent des associations, et certaines expressions révèlent les relations structurelles** (héritage, agrégation, composition).

MÉTHODE Passer de l'énoncé au diagramme de classes

1. Repérer les noms significatifs → classes. Souligne tous les noms communs de l'énoncé. Ne retiens que ceux qui désignent un concept porteur de données ou de comportements (une entité dont on mémorise des informations). Élimine les synonymes et les simples valeurs.
2. Lister les attributs et les méthodes. Pour chaque classe retenue, relève dans l'énoncé

les informations mémorisées (attributs) et les actions que l'objet sait faire (méthodes). Précise la visibilité (+ public, - privé), les types, les paramètres et les types de retour.

3. Repérer les verbes → associations. Chaque verbe reliant deux noms de classes devient une association nommée par ce verbe : « un élève emprunte un livre ».
4. Fixer les multiplicités. Pour chaque extrémité d'association, pose-toi la question : « combien d'objets de l'autre côté sont liés à un objet de ce côté ? ». Traduis la réponse en notation UML : 1, 0..1, 0..* (ou *), 1..*, 0..3, etc.
5. Identifier héritages, agrégations et compositions. Cherche les expressions « est un / est une sorte de » (héritage : flèche à triangle vide), « est composé de / contient » avec vie indépendante (agrégation : losange vide ◊) ou avec destruction solidaire (composition : losange plein ◆).

ATTENTION Ne crée pas une classe pour chaque mot de l'énoncé

L'erreur la plus fréquente consiste à transformer mécaniquement chaque nom commun en classe. Dans la phrase « l'élève donne son nom au bibliothécaire », le mot nom n'est pas une classe : c'est un simple attribut de la classe `Eleve`. Règle pratique : un concept devient une classe uniquement s'il possède plusieurs informations propres ou un comportement. Sinon, c'est un attribut. De même, respecte la contrainte du programme officiel : ton diagramme comportera au plus cinq classes. Si tu en identifies davantage, factorise par héritage (c'est justement à cela que sert la généralisation) ou élimine les concepts secondaires. À l'examen, un diagramme à six classes est sanctionné.

6.2 Étude de cas guidée : la bibliothèque scolaire

Considérons l'énoncé suivant, typique d'une épreuve :

EXEMPLE Énoncé de l'étude de cas

« Le lycée de Nkongsamba veut modéliser sa bibliothèque. Un élève est caractérisé par son matricule, son nom et sa classe ; il peut emprunter au plus 3 livres à la fois. Un livre possède un code ISBN, un titre et un auteur. Chaque emprunt enregistre une date d'emprunt et une date de retour prévue. Le bibliothécaire, qui est comme l'élève une personne (nom, identifiant), enregistre les emprunts. Les livres sont rangés dans des rayons ; un rayon contient plusieurs livres, mais un livre appartient à un seul rayon ; un rayon existe même s'il est momentanément vide. »

Appliquons la démarche, étape par étape.

Étape 1 — Les classes. Les noms significatifs sont : Eleve, Livre, Emprunt, Bibliothecaire, Rayon, Personne. Cela fait six concepts : il faut factoriser. « Le bibliothécaire est, comme l'élève, une personne » : la classe `Personne` devient la **super-classe** d'un héritage. Nous obtenons `Personne`, `Eleve`, `Bibliothecaire`, `Livre`, `Emprunt`, `Rayon` (six classes). **Astuce d'examen** : l'énoncé officiel est calibré pour cinq classes maximum. Ici, pour la richesse de l'exemple, nous conservons `Rayon` (structurelle) et `Emprunt` (classe d'association), mais notez que `Personne`, `Eleve`, `Bibliothecaire` comptent pour trois classes distinctes dans le diagramme. Si la contrainte « 5 max » est stricte, on peut fusionner `Eleve` et `Bibliothecaire` en `Utilisateur` avec un attribut `role`, mais l'héritage est plus élégant et attendu.

Étape 2 — Attributs et méthodes. `Personne` porte `id` et `nom` (communs aux deux sous-classes : on les place une seule fois, dans la super-classe). `Eleve` ajoute `matricule` et `classe`, avec la méthode `emprunter()`. `Livre` porte `isbn`, `titre`, `auteur`. `Emprunt` porte `dateEmprunt` et `dateRetourPrevue`. `Rayon` porte `numero` et `theme`. `Bibliothecaire` ajoute `matriculeAgent` et `enregistrer()`.

Étape 3 — Les associations. Les verbes sont : « emprunter » (entre `Eleve` et `Livre`, matérialisée par la classe d'association `Emprunt`), « enregistrer » (entre `Bibliothecaire` et `Emprunt`), « ranger » (entre `Rayon` et `Livre`).

Étape 4 — Les multiplicités. C'est ici que l'énoncé doit être relu mot par mot.

EXEMPLE Exemple chiffré : traduire « au plus 3 livres »

L'énoncé dit : « un élève peut emprunter au plus 3 livres à la fois ». Question : pour un élève donné, combien de livres sont liés par un emprunt ? Réponse : de 0 (il n'a rien emprunté) à 3. On écrit donc la multiplicité `0..3` du côté de `Livre` (l'extrémité opposée à `Eleve`). Dans l'autre sens : pour un livre donné, combien d'élèves le détiennent à un instant *t* ? Un seul au maximum (il est sur les étagères ou chez un élève) : `0..1` du côté de `Eleve`. Retiens la règle : la multiplicité se note sur l'extrémité opposée à la classe de départ.

De même : un rayon contient plusieurs livres mais peut être vide ($\rightarrow 0..*$ côté `Livre`) ; un livre appartient à un seul rayon ($\rightarrow 1$ côté `Rayon`). Un bibliothécaire enregistre zéro ou plusieurs emprunts (`0..*`) ; un emprunt est enregistré par un seul bibliothécaire (`1`).

Étape 5 — Relations structurelles. « Le bibliothécaire est une personne » et « l'élève est une personne » : deux **héritages** (flèches à triangle vide vers `Personne`). « Les livres sont rangés dans des rayons » et « un rayon existe même s'il est vide » : le rayon et le livre ont des vies indépendantes (détruire un livre ne détruit pas le rayon, et réciproquement) : c'est une **agrégation** (losange vide côté `Rayon`), pas une composition.

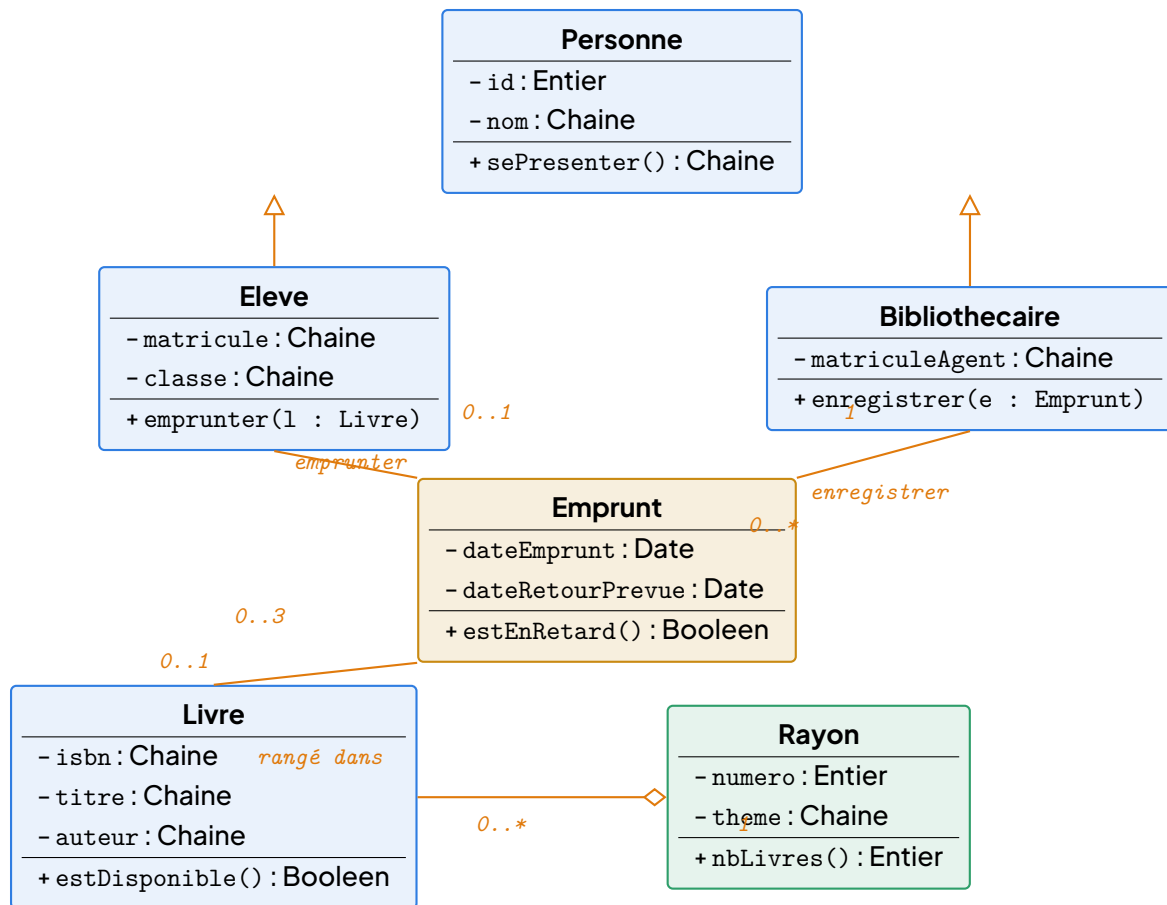


Diagramme de classes de la bibliothèque scolaire : héritage (Personne $\leftarrow$ Eleve, Bibliothecaire), classe Emprunt reliant Eleve et Livre avec multiplicités, agrégation Rayon $\diamond$ Livre.

6.3 Vérifier la cohérence du diagramme

Un diagramme n'est jamais rendu sans relecture. Trois contrôles sont indispensables.

MÉTHODE Vérifier un diagramme de classes avant de le rendre

1. **Connexité** : chaque classe doit être reliée au moins une fois au reste du modèle (association, héritage, agrégation). Une classe isolée est presque toujours le signe d'un oubli.
2. **Relecture des multiplicités dans les deux sens** : pour chaque association, formule deux phrases en français. Exemple : « un élève est lié à 0 à 3 livres » et « un livre est lié à 0 ou 1 élève ». Si une phrase contredit l'énoncé, corrige la multiplicité.
3. **Pas d'attribut dupliqué par héritage** : un attribut commun (comme nom) figure uniquement dans la super-classe Personne. Le recopier dans Eleve et Bibliothecaire est une

erreur de modélisation : l'héritage le transmet déjà automatiquement.

4. Sens des losanges et flèches : l'héritage pointe vers la super-classe ; le losange (agrégation/composition) se place du côté du « tout » (le conteneur).

6.4 Construire le diagramme d'objets associé

Le diagramme de classes décrit le moule (la structure générique) ; le diagramme d'objets en est une photographie à un instant t (une configuration précise d'instances). La démarche est la suivante :

1. Choisir 3 à 5 instances représentatives (une par classe principale suffit souvent).
2. Nommer chaque objet selon la syntaxe UML : `nomObjet : Classe`, le tout souligné.
3. Renseigner les valeurs des attributs (hérités et propres) : `nom = "Amina"`.
4. Tracer les liens : un trait plein simple (sans losange, sans flèche, sans multiplicité) entre deux objets dont les classes sont associées dans le diagramme de classes.

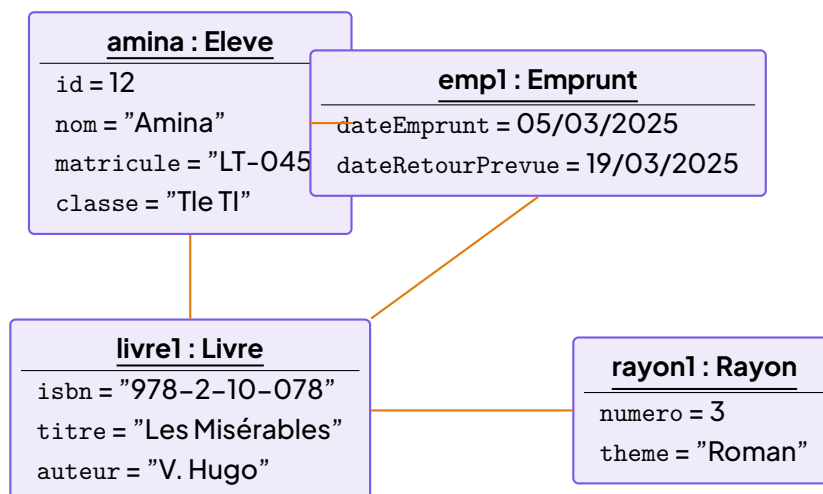


Diagramme d'objets à l'instant t : Amina a emprunté Les Misérables, rangé dans le rayon
 3. Les liens (traits simples) matérialisent les associations du diagramme de classes.

Remarque que l'objet `amina` possède les attributs `id` et `nom` **hérités** de `Personne` : au niveau des instances, l'héritage se traduit par la présence effective des attributs de la super-classe dans l'objet.

EXERCICE RÉSOLU 6 Vérifier et corriger un diagramme

Énoncé. Un élève propose pour la bibliothèque le modèle suivant : la classe `Eleve` contient les attributs `nom`, `matricule` ; la classe `Bibliothecaire` contient aussi `nom` ; l'association `Eleve—Livre` porte les multiplicités 3 côté `Livre` et 1..* côté `Eleve`. L'énoncé disait : « un élève emprunte au plus 3 livres ; un livre est emprunté par au plus un élève à la fois ». Relever trois erreurs et corriger.

SOLUTION

Solution détaillée.

1. Attribut dupliqué : `nom` figure dans `Eleve` et dans `Bibliothecaire`. Correction : créer la super-classe `Personne` portant `nom` (et `id`), et faire hériter les deux classes.
2. Multiplicité 3 côté `Livre` : « au plus 3 » signifie qu'un élève peut n'avoir aucun emprunt. La borne inférieure est 0 : il faut écrire 0..3, pas 3 (qui imposerait exactement trois livres en permanence).
3. Multiplicité 1..* côté `Eleve` : un livre peut être sur les étagères, donc lié à zéro élève, et jamais à plusieurs simultanément : il faut 0..1. La relecture en français — « un livre est emprunté par 0 ou 1 élève » — valide la correction.

LE SAVAIS-TU ? Le diagramme de classes au Baccalauréat TI

Aux épreuves de TI, le diagramme de classes est très souvent noté sur des critères précis : exactitude des notations (losange vide ou plein du bon côté, triangle d'héritage orienté vers la super-classe, visibilité +/–), respect strict des multiplicités de l'énoncé (c'est le critère qui rapporte le plus de points) et lisibilité (classes alignées, traits non croisés autant que possible). Un diagramme juste mais illisible perd des points ; un diagramme élégant avec une multiplicité fausse aussi. Entraîne-toi à relire chaque multiplicité avec une phrase en français : c'est le réflexe des meilleurs candidats.

EXERCICE 5 À toi de jouer : le cybercafé

Un cybercafé de Douala veut modéliser son fonctionnement : un client (nom, téléphone) utilise un poste (numéro, système d'exploitation) pendant une session (heure de début, durée). Un client peut avoir plusieurs sessions, mais un poste n'accueille qu'une session à la fois. Les postes sont regroupés en zones (une zone existe même vide). Produis le diagramme de classes (4 classes maximum) puis un diagramme d'objets avec 3 instances.

CORRIGÉ Exercice : le cybercafé

Diagramme de classes (4 classes) :

- Client (- nom : Chaîne, - telephone : Chaîne ; + demarrerSession()).
- Poste (- numero : Entier, - os : Chaîne).
- Session (- heureDebut : Heure, - duree : Entier).
- Zone (- nom : Chaîne).

Associations :

- Client — Session : multiplicités 1 (côté Session) / 0..* (côté Client). Un client fait 0 ou plusieurs sessions ; une session concerne 1 client.
- Poste — Session : multiplicités 1 (côté Session) / 0..1 (côté Poste). Un poste accueille 0 ou 1 session à la fois ; une session se déroule sur 1 poste.

Agrégation : Zone ◊ — Poste avec 0..* côté Poste et 1 côté Zone (la zone existe même vide, donc agrégation et non composition).

Diagramme d'objets (instant t) :

- jean : Client (nom = "Jean", telephone = "6XX XX XX XX").
- poste5 : Poste (numero = 5, os = "Ubuntu 22.04").
- s1 : Session (heureDebut = "14:00", duree = 120).
- zoneA : Zone (nom = "Zone A").

Liens : jean — s1, poste5 — s1, poste5 — zoneA (traits simples).

À RETENIR L'essentiel de la section

- La construction d'un diagramme suit cinq étapes : noms → classes, attributs/méthodes, verbes → associations, multiplicités, relations structurelles (héritage, agrégation, composition).
- Le programme impose au plus cinq classes : factorise par héritage et élimine les concepts qui ne sont que des attributs.
- Chaque multiplicité se relit en français dans les deux sens ; « au plus 3 » se note 0..3.
- Le diagramme d'objets est une photographie du diagramme de classes : objets soulignés nom : Classe, valeurs d'attributs (y compris héritées), liens en traits simples.
- Au Bac : exactitude des symboles, justesse des multiplicités et lisibilité sont les trois critères de notation.

7 ACTIVITÉ D'INTÉGRATION

8 MODÉLISATION STRUCTURELLE DES SYSTÈMES D'INFORMATION AVEC UML

8.1 Introduction et rôle des diagrammes structurels

DÉFINITION Diagramme structurel UML

Un **diagramme structurel** (ou statique) décrit l'architecture statique d'un système : ses classes, leurs attributs, leurs opérations et les relations qui les lient. Il ne décrit pas le comportement dynamique (séquences, états). En UML 2.x, les principaux diagrammes structurels sont le diagramme de classes (vue logique, générique) et le diagramme d'objets (vue instanciée à un instant t , « photo » du système).

À RETENIR Place dans le cycle de vie

Les diagrammes de classes et d'objets interviennent en phase d'analyse (modélisation du domaine métier) et de conception (architecture logicielle). Un diagramme de classes bien construit se traduit quasi mécaniquement en schéma relationnel (tables, clés primaires/étrangères) pour la base de données — compétence centrale du Module 3.

LE SAVAIS-TU ? UML au Cameroun

L'**OMG** (Object Management Group) standardise UML depuis 1997. Au Cameroun, les grands projets publics (SI de la **CENADI**, portail `impots.cm`, systèmes bancaires **UBA/BCEC**, applications **MTN MoMo/Orange Money**) imposent une modélisation UML rigoureuse avant tout codage. La maîtrise du diagramme de classes (contraintes : ≤ 5 classes au Bac) est un prérequis pour les concours d'entrée aux écoles d'ingénieurs (ENSP, ENSET, POLYTECHNIQUE) et pour la certification **OCP** (Oracle Certified Professional) très prisée des développeurs camerounais.

8.2 Représentation d'une classe

DÉFINITION Classe UML

Une **classe** est un modèle (un type) décrivant un ensemble d'objets partageant les mêmes attributs, opérations, relations et sémantique. Elle se représente par un rectangle à trois compartiments :

1. Nom de la classe (centré, gras, majuscule initiale, ex. Ticket).
2. Attributs (visibilité, nom, type, multiplicité optionnelle, valeur par défaut).
3. Méthodes (visibilité, nom, paramètres typés, type de retour).

PROPRIÉTÉ Syntaxe formelle des attributs et méthodes

- **Attribut** : visibilité nom : Type [multiplicité] = valeurParDéfaut
- **Méthode** : visibilité nom(listeParamètres) : TypeRetour
- Visibilités : + **public** (accessible partout), - **privé** (accessible seulement dans la classe), # **protégé** (accessible dans la classe et ses sous-classes), ~ **paquetage** (par défaut, même paquetage).
- Paramètre : nom : Type (séparés par des virgules). Le type de retour est `void` si aucun retour.

EXEMPLE Classe Ticket — Cantine Lycée Bafoussam

```
// Vue UML équivalente :  
// - numero : int  
// - dateAchat : Date  
// - prix : double  
// + getNumero() : int  
// + getPrix() : double  
// + estValidePour(menu : Menu) : boolean  
  
public class Ticket {  
    private int numero;  
    private Date dateAchat;  
    private double prix; // en FCFA  
  
    public Ticket(int numero, Date dateAchat, double prix) {  
        this.numero = numero;  
        this.dateAchat = dateAchat;  
        this.prix = prix;  
    }  
  
    public int getNumero() { return numero; }  
    public double getPrix() { return prix; }
```

```
public boolean estValidePour(Menu menu) {  
    // Un ticket est valide pour un menu si les dates correspondent  
    return this.dateAchat.equals(menu.getDate());  
}  
}
```

ATTENTION Pièges de syntaxe au Bac

- Oublier les deux points entre le nom et le type (`nom : String`, **pas** `nom String`).
- Confondre `-` (privé) et `#` (protégé) : un attribut privé n'est pas accessible directement dans une sous-classe (héritage). Il faut des accesseurs (`get/set`) ou passer en `#`.
- Écrire le type de retour avant le nom de la méthode (syntax C) au lieu d'après les parenthèses (syntax UML/Java) : `+ calculerTotal() : double`.
- Ne pas mettre `void` pour une procédure sans retour : `+ afficher() : void`.

8.3 Représentation d'un objet

DÉFINITION Objet UML — Diagramme d'objets

Un **objet** est une instance concrète d'une classe à un instant donné. Dans un diagramme d'objets, il se représente par un rectangle souligné : `nomObjet : NomClasse`. Le compartiment attributs affiche des valeurs concrètes : `attribut = valeur`. Les méthodes ne sont pas affichées.

EXEMPLE Objets du mardi 12 mars — Cantine

```
:Eleve  
awa : Eleve  
- matricule = "E102"  
- nom = "Awa"  
- prenom = "Awa"  
- classe = "Tle TI 2"  
  
bala : Eleve  
- matricule = "E117"  
- nom = "Bala"  
- prenom = "Bala"  
- classe = "Tle C"  
  
kamdem : Enseignant
```

```
- matricule = "P07"  
- nom = "Kamdem"  
- prenom = "Martin"  
- matiere = "Informatique"
```

```
t1001 : Ticket  
- numero = 1001  
- dateAchat = 12/03/2025  
- prix = 250.0
```

```
t1002 : Ticket  
- numero = 1002  
- dateAchat = 12/03/2025  
- prix = 250.0
```

```
t1003 : Ticket  
- numero = 1003  
- dateAchat = 12/03/2025  
- prix = 500.0
```

```
menuMardi : Menu  
- date = 12/03/2025  
- platPrincipal = "eru + water fufu"  
- prixRevient = 150.0
```

8.4 Associations et multiplicités

DÉFINITION Association UML

Une **association** représente une relation sémantique forte entre deux classes (ex. « achète », « correspond à »). Elle se dessine par un trait plein. Chaque extrémité porte :

- un rôle (nom du lien vu depuis l'autre classe, ex. acheteur, tickets),
- une multiplicité (cardinalité) indiquant combien d'instances peuvent participer.

PROPRIÉTÉ Multiplicités usuelles

Notation	Sens
1	Exactement un (obligatoire, unique)
0..1	Zéro ou un (optionnel)
* ou 0..*	Zéro ou plusieurs (collection)
1..*	Au moins un (collection non vide)
n..m	Entre n et m inclus (ex. 3..5)

Règle de lecture : La multiplicité se lit à l'extrémité opposée de la classe considérée. Ex. `Personne 1 — 1..* Ticket` : « Une personne achète au moins un ticket » (côté Ticket = 1..*) et « Un ticket appartient à une seule personne » (côté Personne = 1).

EXEMPLE Associations de la cantine

- Achète : `Personne 1 — 1..* Ticket` (rôles : acheteur / tickets).
- Correspond à : `Ticket 1..* — 1 Menu` (rôles : tickets / menuDuJour).

8.5 Héritage, agrégation et composition

DÉFINITION Héritage (Généralisation/Spécialisation)

Relation est-un (**is-a**). L'enfant (**sous-classe**) hérite des attributs et méthodes du parent (**super-classe**) et peut les redéfinir (polymorphisme). Notation : trait plein avec triangle vide pointant vers la super-classe.

DÉFINITION Agrégation

Relation a-un (**has-a**) faible : le tout (**conteneur**) et les parties (**composants**) ont des cycles de vie indépendants. Si le tout est détruit, les parties survivent. Notation : losange vide côté conteneur.

DÉFINITION Composition

Relation est-composé-de forte : le tout et les parties partagent le même cycle de vie. Les parties n'existent pas sans le tout ; si le tout est détruit, les parties le sont aussi. Notation : losange plein côté conteneur.

PROPRIÉTÉ Choix entre Agrégation et Composition — Grille de décision

Critère	Agrégation (losange vide)	Composition (losange plein)
Cycle de vie	Indépendant	Dépendant (cascade delete)
Propriété exclusive	Non (partage possible)	Oui (propriétaire unique)
Exemple classique	Enseignant — Module (un module peut avoir plusieurs enseignants, un enseignant plusieurs modules)	Cantine — Service (si cantine ferme, services disparaissent)

ATTENTION Erreur classique : Agrégation vs Composition

Ne pas confondre « contient » (structurel) et « possède le cycle de vie ». Une `Bibliotheque` **aggrège** des `Livre` (les livres existent si la bibliothèque brûle). Une `Commande` **compose** des `LigneCommande` (une ligne n'a pas de sens sans sa commande). Dans l'énoncé du Bac, cherchez les mots : « n'ont aucune existence propre », « disparaissent avec » → Composition.

8.6 Méthodologie : Construire un diagramme de classes (≤ 5 classes)

MÉTHODE De l'énoncé au diagramme de classes

1. Extraction des noms : Repérer les noms communs métiers (entités) → classes candidates.
2. Filtrage & Fusion : Éliminer les synonymes, fusionner les concepts proches. Compter : si > 5 , arbitrer (factoriser par héritage, supprimer les classes purement techniques, ou créer un second diagramme « zoom »).
3. Attributs & Visibilités : Pour chaque classe, lister attributs (nom, type) → visibilité – par défaut (encapsulation), # si héritage direct.
4. Méthodes : Dédire des règles de gestion (calculs, recherches, changements d'état). Préciser paramètres et retour.
5. Relations : Identifier est-un (héritage), a-un (association/agrégation/composition). Poser les multiplicités justifiées par une phrase métier.
6. Dessin : Papier millimétré, règle, crayon. Classes en rectangles 3 compartiments. Relations alignées horizontalement/verticalement. Légender les associations.
7. Vérification : Chaque règle de gestion est-elle couverte ? Contrainte ≤ 5 classes respectée ?

8.7 Activité d'intégration : La cantine du Lycée de Bafoussam

8.8 Objectifs de l'activité

À l'issue de cette activité, tu seras capable de :

- Analyser un énoncé réel pour en dégager les **classes** candidates, **attributs**, **méthodes**.
- Représenter une classe UML avec **visibilités** (+, -, #), types, paramètres, retours.
- Matérialiser **association/multiplicités**, **héritage**, **agrégation**, **composition** avec justifications métier.
- Produire un **diagramme de classes** complet (≤ 5 classes) et un **diagramme d'objets** cohérent.
- Argumenter oralement chaque choix de modélisation (compétence **savoir-être**).

8.9 Situation professionnelle

Le Lycée de Bafoussam (1 450 élèves, 62 enseignants) souhaite informatiser sa cantine (600 repas/jour). Actuellement : tickets papier, caisse espèces, cahier manuel → erreurs, pertes, pas de prévision menus.

Règles de gestion (recueillies auprès de l'économe) :

1. La cantine sert des **élèves** et des **enseignants**. Toute personne : **matricule**, **nom**, **prénom**. Élève + **classe** ; Enseignant + **matière**.
2. Une personne **achète un ou plusieurs tickets** (achat anticipé possible). Un ticket : **numéro unique**, **date achat**, **prix** (250 FCFA élève, 500 FCFA enseignant). Un ticket = une seule personne.
3. Chaque ticket **correspond à un menu du jour** (date, plat principal, prix de revient). Un menu = plusieurs tickets (ceux du jour) ; un ticket = un seul menu.
4. La cantine = 3 **services** : **Caisse**, **Cuisine**, **Distribution**. **Aucune existence propre** : si cantine ferme, services disparaissent. Chaque service : **nom**, **responsable**.
5. L'économe veut connaître le **nombre de tickets vendus pour un menu donné** (prévoir quantités).

Contrainte Bac : Diagramme de classes ≤ 5 classes. Analyse initiale : 6 candidats (Personne, Eleve, Enseignant, Ticket, Menu, Cantine/Service). Arbitrage justifié obligatoire.

8.10 Consignes de travail (Groupes de 4, 2 h)

1. **Classes candidates** : Lister avec citation de la règle justificative. Expliquer le passage 6 → 5 sans perte d'info.
2. **Attributs & Visibilités** : Tableau par classe (nom, type, visibilité). Justifier - (encapsulation).
3. **Méthodes** : ≥ 1 par classe (params, retour). Impératif : méthode Menu : nbTicketsVendus()

(règle 5).

4. **Héritage** : Représenter `Personne` ← `Eleve`, `Enseignant`. Justifier. Visibilité attributs `Personne` pour accès sous-classes?
5. **Associations** : « achète » (`Personne`–`Ticket`), « correspond à » (`Ticket`–`Menu`). Multiplicités aux deux bouts + justification phrase française.
6. **Composition** : `Cantine`–`Services`. Choix agrégation/composition + justification règle 4.
7. **Diagramme de classes** : Assembler (5 classes max). Vérifier couverture règles 1–5.
8. **Diagramme d'objets** : Mardi 12 mars. Objets : `awa` (E102, Tle TI 2), `bala` (E117, Tle C), `kamdem` (P07, Informatique), tickets 1001/1002/1003, menu « `eru` + `water fufu` ». Vérifier multiplicités.
9. **Confrontation** : Échanger avec un autre groupe. Identifier ≥ 1 différence (multiplicité/relation). Décider ensemble la meilleure option, justifier oralement.

8.11 Ressources à mobiliser

À RETENIR

Mémento UML — Syntaxe Bac

- **Classe** : 3 compartiments. **Attribut** : visibilité nom : Type. **Méthode** : visibilité nom(param : Type) : TypeRetour.
- **Visibilités** : + public, – privé, # protégé (sous-classes).
- **Objet** : Nom souligné nomObjet : Classe. **Attributs** = valeurs.
- **Multiplicités** : 1, 0..1, * (0..*), 1..*. Se lit à l'extrémité opposée.
- **Héritage** : Trait plein + triangle vide vers parent.
- **Agrégation** : Losange vide côté conteneur.
- **Composition** : Losange plein côté conteneur.

8.12 Démarche et corrigé commenté

EXERCICE RÉSOLU 7

Modélisation complète — Cantine Lycée Bafoussam

Énoncé : Produire le diagramme de classes (5 classes max) et le diagramme d'objets du mardi 12 mars, en mobilisant héritage, associations, multiplicités, composition.

SOLUTION

Étape 1 — Arbitrage des 5 classes (Contrainte officielle).

- **Candidats bruts** : Personne, Eleve, Enseignant, Ticket, Menu, Cantine, Service.
- **Choix retenu (cœur métier prioritaire)** : Les 5 classes du diagramme principal sont :

Personne, Eleve, Enseignant, Ticket, Menu

- **Justification** : L'héritage Eleve/Enseignant → Personne structure le domaine « usager » (règle 1). Ticket et Menu couvrent les règles 2, 3, 5 (vente, liaison, prévision). La règle 4 (Cantine/Services) est une contrainte organisationnelle contexte : elle sera modélisée dans un diagramme complémentaire « Zoom Cantine » (2 classes : Cantine ♦-Service, multiplicité 3) pour respecter la limite de 5 sans perdre l'information. Tout choix argumenté (ex. fusionner Service dans Cantine via attribut multivalué) est recevable au Bac s'il est justifié.

Étape 2 — Attributs, visibilité, méthodes (Niveau modèle).

```
<<abstract>> Personne
# matricule : String
# nom : String
# prenom : String
+ sePresenter() : String
+ getMatricule() : String
+ prixTicket() : double // {abstract} : 250 ou 500 selon sous-classe

Eleve
- classe : String // ex. "Tle TI 2"
+ prixTicket() : double // return 250.0
+ getClasse() : String

Enseignant
- matiere : String // ex. "Informatique"
+ prixTicket() : double // return 500.0
+ getMatiere() : String

Ticket
- numero : int
- dateAchat : Date
- prix : double // FCFA, figé à la création selon type acheteur
+ getNumero() : int
+ getDateAchat() : Date
+ getPrix() : double
+ estValidePour(unMenu : Menu) : boolean

Menu
- date : Date
- platPrincipal : String
- prixRevient : double // Coût matière première
+ getDate() : Date
+ getPlatPrincipal() : String
+ getPrixRevient() : double
+ nbTicketsVendus() : int // Règle 5 : compte tickets liés
```

ATTENTION**Justification des visibilité**

- **Personne** : attributs # (protégé) pour que **Eleve** et **Enseignant** y accèdent directement (héritage). Méthodes + (interface publique).
- **Eleve**, **Enseignant**, **Ticket**, **Menu** : attributs - (privé) → encapsulation (masquage implémentation, contrôle via accesseurs `get/set`). C'est la règle par défaut en conception objet.

Étape 3 — Relations et multiplicités (Justification métier).

Relation	Notation UML	Justification (Règle de gestion)
Héritage	Eleve - > Personne	« Un élève est une personne » (R1). Même pour Enseignant.
Association	Personne 1 - 1..* Ticket	Côté Ticket (1..*) : « Une personne achète au moins un ticket » (R2). Côté Personne (1) : « Un ticket appartient à une seule personne » (R2).
Association	Ticket 1..* - 1 Menu	Côté Menu (1) : « Un ticket vaut pour un seul menu » (R3). Côté Ticket (1..*) : « Un menu correspond à plusieurs tickets (ceux du jour) » (R3).
Composition	Cantine ♦ 1 - 3 Service	« Les services n'ont aucune existence propre : si la cantine ferme, ils disparaissent » (R4) → Composition (losange plein), multiplicité fixe 3 (Caisse, Cuisine, Distribution).

Étape 4 — Diagramme de classes principal (5 classes).

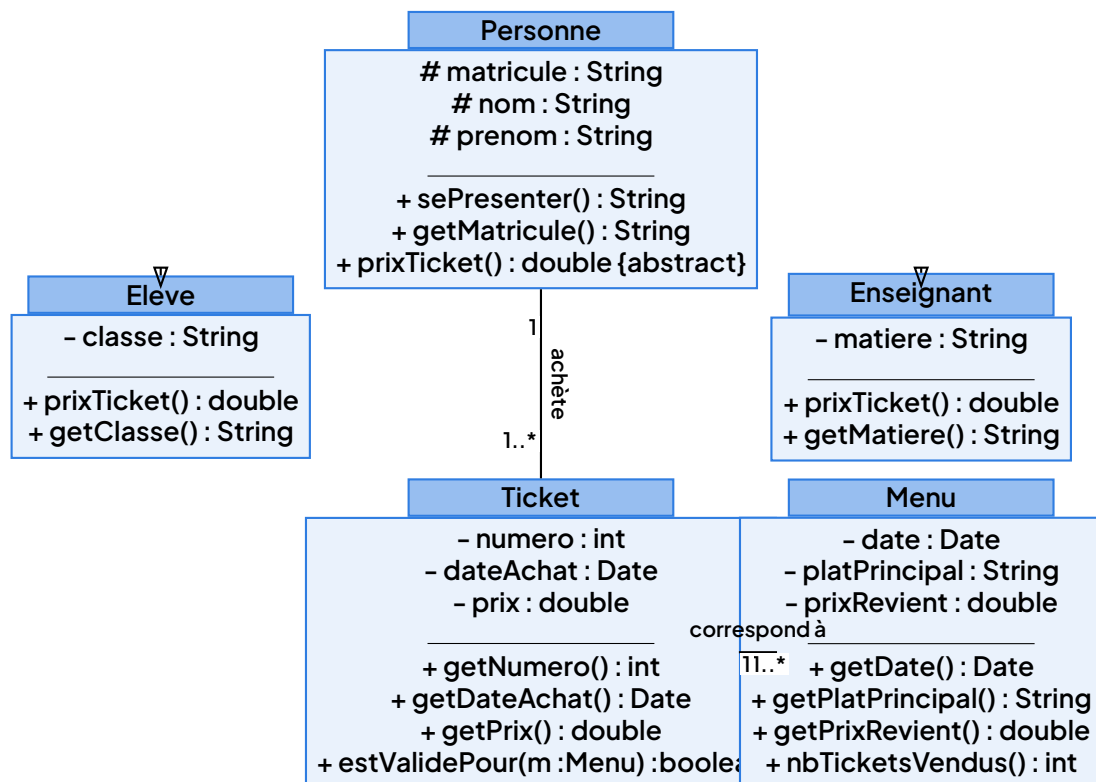
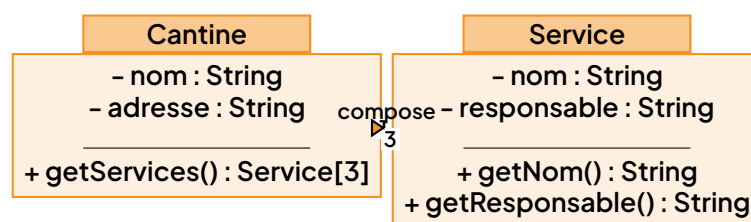


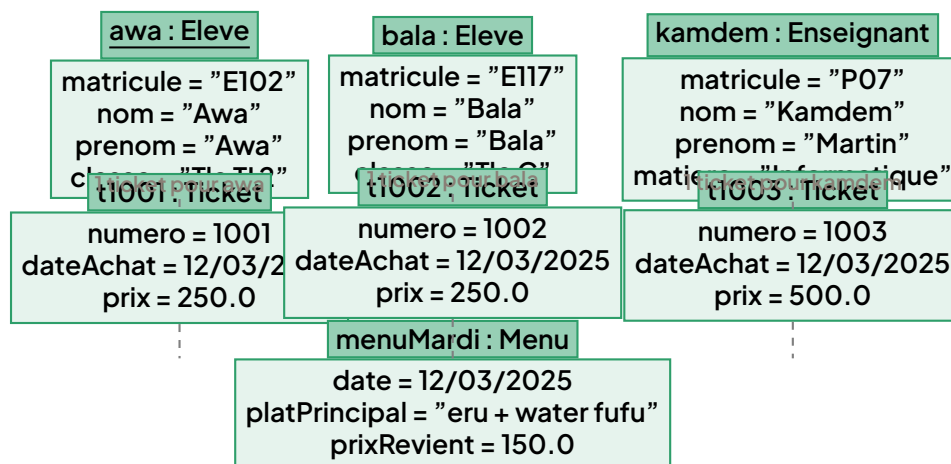
Diagramme de classes principal (5 classes) : Héritage, Associations avec multiplicités et rôles.

Étape 5 — Diagramme complémentaire : Composition Cantine–Service (Règle 4).



Zoom sur la composition Cantine–Service (Règle 4). Losange plein côté Cantine, multiplicité 3 fixe.

Étape 6 — Diagramme d’objets : Mardi 12 mars 2025 (Instantané).



3 tickets pour ce menu (1..* côté Ticket, 1 côté Menu)

Diagramme d'objets du mardi 12 mars. Noms soulignés, valeurs concrètes, liens respectant les multiplicités du diagramme de classes (1 personne → 1..* tickets; 1..* tickets → 1 menu).

Étape 7 — Vérification de la couverture des règles de gestion.

Règle	Élément UML	Couverture
R1 (Personnes, élève/enseignant)	Héritage $\text{Personne} \leftarrow \text{Eleve, Enseignant}$ + attributs classe/matiere	OK
R2 (Achat tickets, prix différencié)	Assoc. $\text{Personne} - \text{Ticket}$ (mult 1/1..*) + méthode $\text{prixTicket}()$ polymorphique	OK
R3 (Ticket ↔ Menu)	Assoc. $\text{Ticket} - \text{Menu}$ (mult 1..*/1) + attributs date, platPrincipal	OK
R4 (Cantine = 3 services, cycle vie)	Diagramme complémentaire : Composition $\text{Cantine} \blacklozenge - \text{Service}$ (mult 3)	OK
R5 (Nb tickets vendus / menu)	Méthode $\text{Menu} : \text{nbTicketsVendus}() : \text{int}$	OK

Étape 8 — Points de vigilance pour la confrontation (Consigne 9).

- Multiplicité Ticket–Menu : Certains groupes mettent 0..* côté Ticket (ticket peut ne

pas être utilisé?). L'énoncé dit « Chaque ticket correspond à un menu » → 1..* obligatoire côté Ticket.

- Visibilité Personne : Si attributs – (privé), Eleve ne peut pas accéder à matricule sans getters. Solution : # (protégé) ou getters + dans Personne.
- Composition vs Agrégation : « Services n'ont aucune existence propre » → Composition (losange plein). Agrégation (vide) = erreur.
- Nom d'objet : Doit être souligné (awa : Eleve). Non souligné = erreur de syntaxe objet.

ATTENTION Erreurs fréquentes récapitulées (Pour le Bac)

- Multiplicités inversées : Lire toujours « depuis l'autre classe ». « Une personne achète des tickets » → multiplicité 1..* côté Ticket.
- Losange vide pour composition : Règle 4 → losange plein (noir). Losange vide = agrégation = 0 point.
- Oubli du soulignement sur les noms d'objets (awa : Eleve pas awa : Eleve).
- Attributs privés dans super-classe sans accesseurs : bloque l'héritage pratique. Utiliser # ou + getX().
- Plus de 5 classes sur le diagramme principal : Échec contrainte. Faire un diagramme annexe (zoom) pour le reste.
- Méthode sans type de retour : + calculer() invalide. Obligatoire : void ou : Type.
- Paramètres sans type : + ajouter(x) invalide. Obligatoire + ajouter(x : Type).

8.13 Exercice d'entraînement individuel

EXERCICE 6 Gestion d'un Cybercafé à Yaoundé — 20 min

Le gérant du cybercafé « NetPlus » (quartier Mvog-Ada) veut informatiser. Règles :

1. Clients : Occasionnels (paient à l'heure : 200 FCFA/h) et Abonnés (forfait mensuel 5 000 FCFA, identifiés par login/password). Tout client a id, nom, tel.
2. Un client utilise un Poste (PC) pour une Session. Un poste = 1 session à la fois. Une session = 1 poste, 1 client, debut, fin, cout.
3. Le cybercafé contient 20 postes fixes (numérotés 1..20, config, etat). Si le cyber ferme, les postes restent (matériel).
4. Le gérant veut lister les sessions d'un client sur une période.

Travail : 1) Classes candidates → 5 max (justifier). 2) Diagramme de classes (héritage, assoc, mult, agrégation/composition). 3) Méthode `Client::getSessions(periode)`. 4) Diagramme d'objets : Client abonné « Jean » (id=1) en session sur Poste 5 le 20/05.

CORRIGÉ Exercice Cybercafé NetPlus

1. Classes (5) : Client (abstraite), Occasionnel, Abonne, Poste, Session. (Cybercafé géré à part : composition/agrégation? Postes survivent → Agrégation Cybercafé ◊- Poste). 2. Héritage : Occasionnel/Abonne -|> Client. Client # id, nom, tel. 3. Associations :

- Client 1 - 0..* Session (rôle client / sessions). « Un client a 0 ou plusieurs sessions ».
- Poste 1 - 0..1 Session (rôle poste / sessionActuelle). « Un poste a 0 ou 1 session en cours ».
- Session 1 - 1 Client et 1 - 1 Poste (liens forts).

4. Agrégation : Cybercafé ◊ (vide) - 20 Poste. (Postes existent sans cyber). 5. Méthode : + `getSessions(debut:Date, fin:Date) : List<Session>` dans Client. 6. Objets : jean : Abonne (id=1, login="jean", pwd="***"), poste5 : Poste (num=5, etat="Occupé"), sess1 : Session (debut=20/05 10h, fin=null, cout=0, lien vers jean et poste5). Multiplicités respectées.

8.14 Bilan et perspectives

À RETENIR Ce qu'il faut retenir pour l'épreuve

- Diagramme de classes = Modèle statique (types). Diagramme d'objets = Instantané (instances, valeurs, liens concrets).
- 5 classes max : Contrainte dure. Arbitrer par factorisation (héritage) ou diagramme complémentaire. Justifier par écrit sur la copie.
- Visibilités : - par défaut (encapsulation), # pour héritage, + pour services (API).
- Multiplicités : Les écrire aux deux extrémités. Justifier chaque chiffre par une phrase de la règle de gestion.
- Composition vs Agrégation : Mots-clés « disparaissent avec », « aucune existence propre » → Composition (losange plein). Sinon → Agrégation (vide).
- Passage au relationnel : Classe → Table. Attributs → Colonnes. Héritage → Table par classe (ou table unique avec discriminateur). Association 1..* → Clé étrangère côté *. Composition → Clé étrangère NOT NULL + ON DELETE CASCADE.

LESAVAIS-TU ? Poursuite d'études

En BTS/DUT/Ingénieur (ENSP, IUT Douala, POLYTECHNIQUE), vous utiliserez des outils CASE (StarUML, Visual Paradigm, PlantUML, Mermaid) pour générer le code (Java, PHP, C#, Python) et le SQL (DDL) directement depuis le diagramme de classes. La rigueur acquise ici (visibilités, multiplicités, composition) évite des bugs coûteux en production (ex. suppression en cascade non voulue, références circulaires JSON, failles d'injection par manque d'encapsulation).

9

MÉTHODES ET EXERCICES RÉSOLUS

9.1 Méthode 1 : Représenter une classe dans le formalisme UML

MÉTHODE

Construire le rectangle d'une classe en trois compartiments

Pour représenter correctement une classe UML, suis toujours la même démarche :

1. Identifier le nom de la classe : un nom commun singulier, avec une majuscule initiale (convention) : Client, Compte, Livre. Il occupe le **premier compartiment**.
2. Recenser les attributs (les données) dans le **deuxième compartiment**, un par ligne, avec la syntaxe :

visibilité nomAttribut : Type

La visibilité se note : + (public), - (privé), # (protégé). En conception rigoureuse, les attributs sont presque toujours privés (-) : c'est le principe d'**encapsulation**.

3. Recenser les méthodes (les opérations) dans le **troisième compartiment**, avec la syntaxe complète :

visibilité nomMethode(paramètre : Type) : TypeRetour

Les méthodes sont généralement publiques (+) car elles forment l'interface de la classe. Une méthode qui ne retourne rien s'écrit sans type de retour (ou : void).

4. Vérifier la cohérence : chaque attribut manipulé par une méthode doit exister dans le deuxième compartiment ; chaque paramètre doit avoir un type.

Réflexe Bac : si l'énoncé donne un dictionnaire de données ou une description textuelle, souligne les noms (attributs) et les verbes d'action (méthodes) avant de dessiner.

EXERCICE RÉSOLU 8

Modéliser la classe CompteBancaire

Une agence UBA de Yaoundé veut modéliser ses comptes. Un compte bancaire est caractérisé par un numéro (chaîne de caractères), un solde (réel) et le nom du titulaire (chaîne). On doit pouvoir consulter le solde, déposer un montant et retirer un montant (le retrait modifie le solde et retourne un booléen indiquant si l'opération a réussi). Représenter la classe `CompteBancaire` dans le formalisme UML, en respectant l'encapsulation.

SOLUTION

Étape 1 — Nom de la classe. L'énoncé l'impose : `CompteBancaire`.

Étape 2 — Attributs. Les noms caractéristiques sont : numéro, solde, titulaire. Par encapsulation, ils sont privés (-) :

- - numero : Chaine
- - solde : Reel
- - titulaire : Chaine

Étape 3 — Méthodes. Les verbes d'action de l'énoncé donnent trois opérations publiques (+):

- « consulter le solde » : pas de paramètre, retourne un réel $\Rightarrow$ + consulterSolde() : Reel
- « déposer un montant » : un paramètre réel, pas de retour $\Rightarrow$ + déposer(montant : Reel)
- « retirer un montant » : un paramètre réel, retourne un booléen $\Rightarrow$ + retirer(montant : Reel) : Booleen

Étape 4 — Représentation UML.

CompteBancaire
- numero : Chaine - solde : Reel - titulaire : Chaine
+ consulterSolde() : Reel + déposer(montant : Reel) + retirer(montant : Reel) : Booleen

La classe `CompteBancaire` : trois compartiments (nom, attributs privés, méthodes publiques).

Conclusion. La classe `CompteBancaire` possède trois attributs privés et trois méthodes publiques ; chaque méthode précise ses paramètres typés et son type de retour, conformément au formalisme UML.

9.2 Méthode 2 : Représenter un objet (instance de classe)

MÉTHODE Dessiner un objet UML correctement

Un **objet** est une instance concrète d'une classe. Pour le représenter :

1. Dessine un rectangle à deux compartiments (jamais trois : un objet n'a pas de méthodes propres, il utilise celles de sa classe).
2. Dans le premier compartiment, écris l'identité de l'objet sous la forme `nomObjet : NomClasse`, soulignée (c'est le signe graphique officiel d'une instance en UML). On

peut aussi écrire : `NomClasse` (objet anonyme).

3. Dans le deuxième compartiment, donne la valeur de chaque attribut : `nomAttribut = valeur`. Les valeurs textuelles s'écrivent entre guillemets.
4. Vérifie que chaque attribut listé existe bien dans la classe et que la valeur respecte le type déclaré (un réel pour `solde`, une chaîne pour `titulaire`).

Réflexe Bac : un diagramme d'objets est une photographie du système à un instant donné ; plusieurs objets d'une même classe peuvent coexister avec des valeurs différentes.

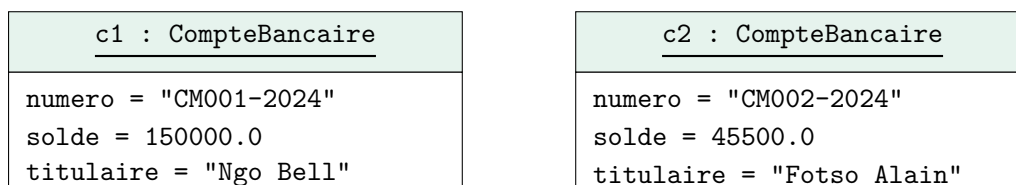
EXERCICE RÉSOLU 9 Instancier deux objets de la classe `CompteBancaire`

À partir de la classe `CompteBancaire` de l'exercice précédent, représenter dans le formalisme UML les deux objets suivants, extraits du fichier clients de l'agence :

- le compte `c1` : numéro « CM001-2024 », solde 150 000 FCFA, titulaire « Ngo Bell » ;
- le compte `c2` : numéro « CM002-2024 », solde 45 500 FCFA, titulaire « Fotso Alain ».

SOLUTION

Raisonnement. Un objet se note `nom : Classe` souligné, suivi des valeurs des attributs. Les soldes sont des réels (on peut écrire `150000.0`), les numéros et titulaires sont des chaînes entre guillemets. Aucun compartiment de méthodes n'apparaît.



Deux instances de la classe `CompteBancaire` : en-têtes soulignés, valeurs conformes aux types.

Vérification. Chaque attribut (`numero`, `solde`, `titulaire`) existe dans la classe ; chaque valeur respecte le type déclaré ; les en-têtes sont soulignés.

Conclusion. `c1` et `c2` sont deux objets distincts de la même classe : ils partagent la même structure (attributs et méthodes) mais portent des valeurs différentes — c'est exactement la différence entre une classe (moule) et un objet (instance).

9.3 Méthode 3 : Matérialiser une association et sa multiplicité

MÉTHODE Tracer une association avec multiplicités

1. Repérer l'association dans l'énoncé : elle correspond à un verbe reliant deux classes (« un client passe une commande », « un élève s'inscrit à un club »).
2. Tracer une ligne pleine simple (sans flèche ni losange) entre les deux classes. On peut nommer l'association par le verbe au milieu de la ligne.
3. Traduire chaque phrase de cardinalité en multiplicité UML, placée à l'extrémité opposée de la classe concernée :

Multiplicité	Signification
1	exactement un (obligatoire et unique)
0..1	zéro ou un (optionnel)
* ou 0..*	zéro, un ou plusieurs
1..*	au moins un

4. Lecture croisée (point le plus délicat) : la multiplicité placée du côté de la classe B indique combien d'objets B sont liés à UN objet A. Formule à retenir : « un A est associé à (multiplicité côté B) B ».
5. Vérifie en relisant les deux phrases : « un A → combien de B ? » et « un B → combien de A ? ».

EXERCICE RÉSOLU 10 Association Client–Commande pour une boutique en ligne

La startup camerounaise Shop237 veut modéliser ses ventes. L'énoncé précise : « un client peut passer plusieurs commandes, mais peut aussi être inscrit sans avoir encore commandé ; chaque commande est passée par un seul client ». Représenter l'association entre les classes `Client` et `Commande` avec les multiplicités exactes.

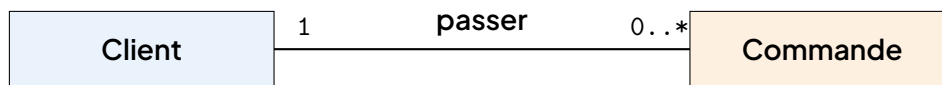
SOLUTION

Étape 1 — Le verbe. « passer » relie `Client` et `Commande` : c'est l'association passer.

Étape 2 — Multiplicité côté `Commande`. Question : « UN client est associé à combien de commandes ? » L'énoncé dit : plusieurs commandes possibles, mais zéro accepté (client inscrit sans commande). Donc : 0..* (ou *), placé du côté de `Commande`.

Étape 3 — Multiplicité côté `Client`. Question : « UNE commande est associée à combien de clients ? » L'énoncé dit : « un seul client ». Donc : 1, placé du côté de `Client`.

Étape 4 — Schéma.



Association passer : un client passe zéro ou plusieurs commandes ; chaque commande est passée par exactement un client.

Vérification croisée. Lecture côté Commande : « un client → 0..* commandes » ✓. Lecture côté Client : « une commande → 1 client » ✓.

Conclusion. L'association passer est une association un-à-plusieurs de multiplicités 1 (côté Client) et 0..* (côté Commande) ; le piège classique — inverser les multiplicités — est évité en appliquant la lecture croisée.

9.4 Méthode 4 : Matérialiser un héritage

MÉTHODE Reconnaître et dessiner une relation d'héritage

1. Repérer l'héritage : il apparaît quand une classe « est une sorte de » une autre (« un épargnant est un client », « un enseignant est une personne »). La classe générale est la **super-classe** (ou classe mère) ; la classe spécialisée est la **sous-classe** (classe fille).
2. Factoriser : place dans la super-classe tous les attributs et méthodes communs ; ne laisse dans la sous-classe que ce qui est spécifique. Ne recopie jamais un attribut hérité dans la sous-classe.
3. Dessiner la flèche : une ligne pleine terminée par un triangle blanc (non rempli) pointant vers la super-classe. La flèche part de la sous-classe et monte vers la classe mère.
4. Visibilité protégée : si un attribut de la super-classe doit être accessible aux sous-classes, déclare-le # (protégé) plutôt que - (privé).
5. Vérifie : la sous-classe hérite de tout le contenu public et protégé de la super-classe, et peut ajouter ses propres attributs et méthodes.

Réflexe Bac : le sens de la flèche est le piège n°1 — le triangle désigne toujours la classe mère.

EXERCICE RÉSOLU 11 Hiérarchie de comptes pour une microfinance

Une microfinance de Douala gère deux types de comptes. Tout compte possède un numéro et un solde, et permet de consulter le solde. Un `CompteEpargne` possède en plus un

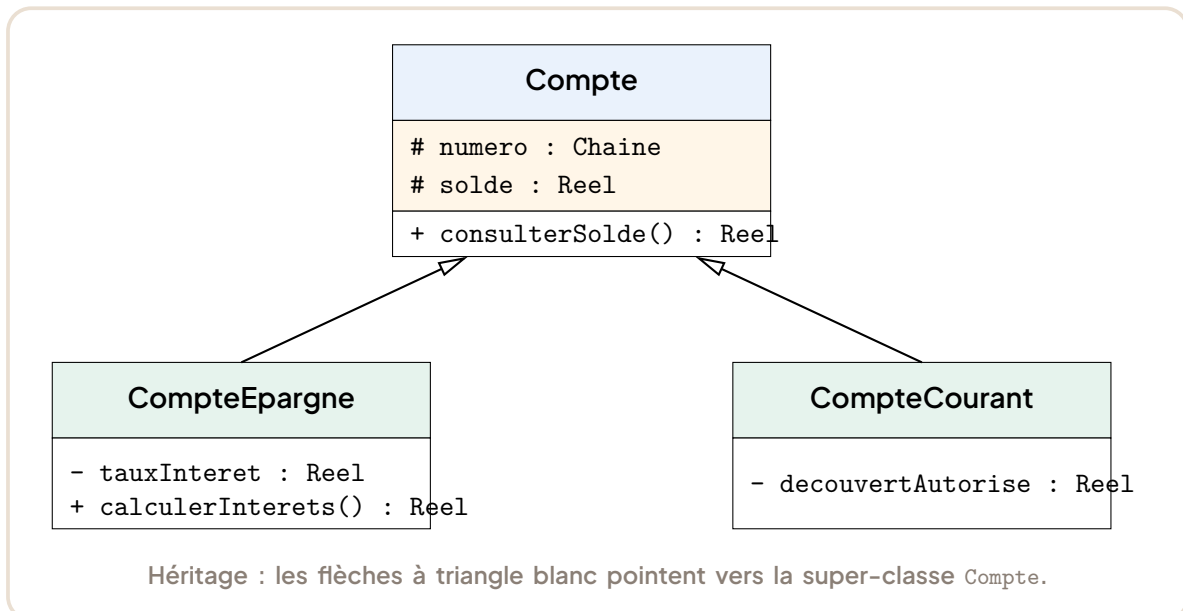
taux d'intérêt (réel) et une méthode `calculerInterets() : Reel`. Un `CompteCourant` possède en plus un découvert autorisé (réel). Modéliser cette situation par un diagramme utilisant l'héritage.

SOLUTION

Étape 1 — « est une sorte de ». Un compte épargne est un compte ; un compte courant est un compte. Il y a donc une super-classe `Compte` et deux sous-classes.

Étape 2 — Factorisation. Commun : `numero`, `solde`, `consulterSolde()` ⇒ dans `Compte`. Spécifique à l'épargne : `tauxInteret`, `calculerInterets()`. Spécifique au courant : `decouvertAutorise`. Les attributs de la super-classe sont déclarés # (protégés) pour être accessibles aux sous-classes.

Étape 3 — Schéma.



Vérification. `numero` et `solde` ne figurent que dans `Compte` : ils sont hérités, non recopiés. Les triangles blancs pointent bien vers la classe mère.

Conclusion. `CompteEpargne` et `CompteCourant` héritent des attributs protégés `numero`, `solde` et de la méthode `consulterSolde()` de `Compte`, tout en ajoutant leurs caractéristiques propres : l'héritage évite toute duplication.

9.5 Méthode 5 : Distinguer et dessiner agrégation et composition

MÉTHODE Choisir entre agrégation et composition

Les deux relations expriment « est composé de », mais avec des forces différentes. Pour choisir :

1. Pose la question du cycle de vie : « si le contenant est détruit, la partie peut-elle exister seule ? »
 - Oui (la partie survit, peut être partagée) ⇒ **agrégation** : losange blanc (vide) côté contenant. Exemple : une classe scolaire et ses élèves — si la classe est dissoute, les élèves existent toujours.
 - Non (la partie meurt avec le tout, n'appartient qu'à un seul tout) ⇒ **composition** : losange noir (plein) côté contenant. Exemple : une commande et ses lignes de commande — supprimer la commande supprime ses lignes.
2. Dessine une ligne pleine entre les deux classes, avec le losange du côté du contenant (le « tout »), jamais du côté de la partie.
3. Ajoute les multiplicités comme pour une association ordinaire (souvent 1 côté contenant, 1..* côté partie).

Formule à retenir : agrégation = losange vide + partie autonome ; composition = losange plein + partie liée au destin du tout.

EXERCICE RÉSOLU 12 Agrégation ou composition ? Le cas de la bibliothèque scolaire

Pour le logiciel de gestion de la bibliothèque d'un lycée de Bafoussam, on hésite sur deux relations :

1. entre **Bibliothèque** et **Rayon** : un rayon est créé avec la bibliothèque et n'a aucun sens sans elle ;
2. entre **Rayon** et **Livre** : un livre rangé dans un rayon peut être déplacé vers un autre rayon, et continue d'exister si le rayon est supprimé.

Pour chaque relation, justifier le choix (agrégation ou composition) et représenter le diagramme de classes correspondant.

SOLUTION

Analyse de la relation 1 (Bibliothèque–Rayon). Un rayon « n'a aucun sens sans » la bibliothèque : sa destruction suit celle du tout. De plus, un rayon appartient à une seule bibliothèque. Il s'agit donc d'une composition : losange plein du côté de **Bibliothèque**.

Analyse de la relation 2 (Rayon–Livre). Un livre « continue d'exister si le rayon est supprimé » et peut changer de rayon : la partie est autonome. Il s'agit donc d'une agrégation : losange vide du côté de **Rayon**.

Schéma.



Composition (losange plein) entre Bibliotheque et Rayon; agrégation (losange vide) entre Rayon et Livre.

Conclusion. La relation Bibliotheque-Rayon est une composition car le cycle de vie du rayon dépend de celui de la bibliothèque ; la relation Rayon-Livre est une agrégation car le livre existe indépendamment de son rayon. Le critère décisif est toujours l'autonomie de la partie.

9.6 Méthode 6 : Produire un diagramme de classes complet (au plus cinq classes)

MÉTHODE Construire un diagramme de classes à partir d'un énoncé

1. Repérer les classes : souligne les noms communs principaux de l'énoncé (au plus cinq au Bac). Écarte les noms qui sont en réalité des attributs (ex. « nom de l'élève » n'est pas une classe).
2. Pour chaque classe, liste attributs (privés, typés) et méthodes (publiques, paramètres et retours typés) — applique la méthode 1.
3. Repérer les relations : verbes entre classes $\Rightarrow$ associations avec multiplicités ; « est une sorte de » $\Rightarrow$ héritage ; « est composé de » $\Rightarrow$ agrégation ou composition (test du cycle de vie).
4. Placer les multiplicités par lecture croisée, puis relire chaque relation dans les deux sens.
5. Mise en page : super-classes en haut, classes liées proches, lignes qui ne se croisent pas si possible. Un diagramme lisible et complet (aucune classe isolée sans raison) vaut plus qu'un diagramme chargé.

EXERCICE RÉSOLU 13 Diagramme de classes d'un système de gestion de notes

Le proviseur d'un lycée de Ngaoundéré veut un système de gestion des notes. Règles de gestion :

- un élève est caractérisé par son matricule, son nom et sa date de naissance ; on peut calculer sa moyenne ;
- une classe (ex. Tle TI 2) a un nom et regroupe au moins un élève ; un élève appartient à

une seule classe ;

- un enseignant (nom, spécialité) enseigne une ou plusieurs matières ; une matière (intitulé, coefficient) est enseignée par un seul enseignant ;
- une évaluation (date, note sur 20) concerne un seul élève et une seule matière ; un élève peut avoir plusieurs évaluations.

Produire le diagramme de classes complet (cinq classes).

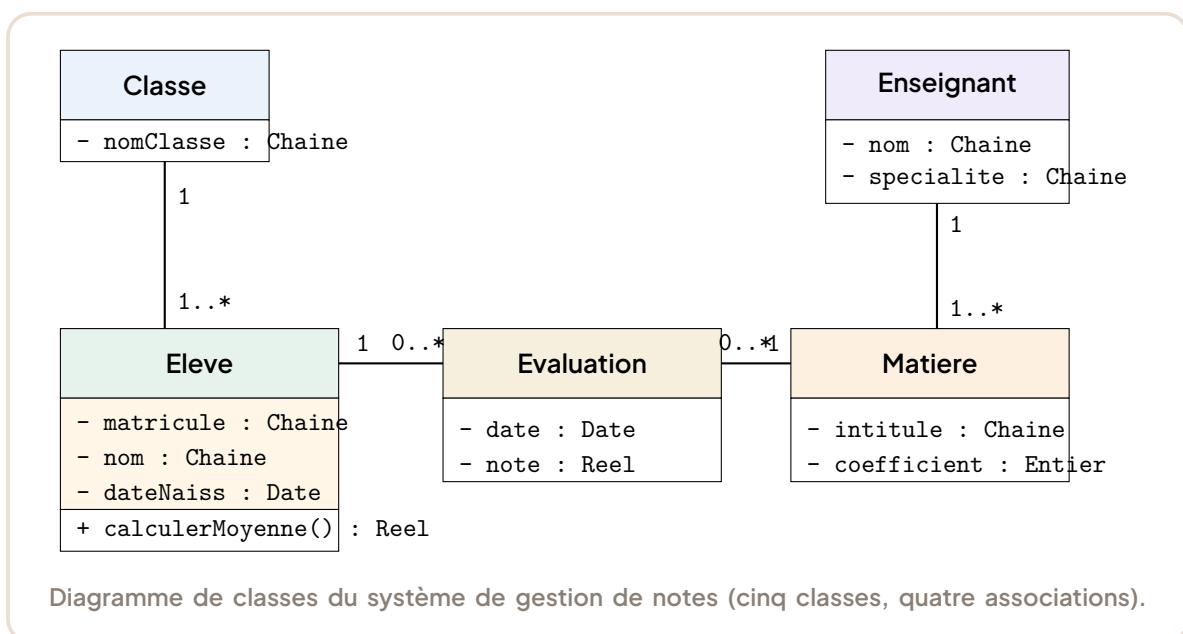
SOLUTION

Étape 1 — Les cinq classes. Eleve, Classe, Enseignant, Matiere, Evaluation. (« Matricule », « nom », « note » sont des attributs, pas des classes.)

Étape 2 — Relations et multiplicités (lecture croisée).

- Classe-Eleve : « une classe regroupe au moins un élève » $\Rightarrow$ 1..* côté Eleve ; « un élève appartient à une seule classe » $\Rightarrow$ 1 côté Classe. L'énoncé ne lie pas la destruction des élèves à celle de la classe : on reste sur une association simple.
- Enseignant-Matiere : 1..* côté Matiere, 1 côté Enseignant.
- Eleve-Evaluation : 0..* côté Evaluation (« peut avoir plusieurs », zéro possible), 1 côté Eleve.
- Matiere-Evaluation : 0..* côté Evaluation, 1 côté Matiere.

Étape 3 — Diagramme.



Vérification finale. Cinq classes maximum ✓ ; attributs privés et typés ✓ ; méthode calculerMoyenne() avec type de retour ✓ ; chaque multiplicité relue dans les deux sens ✓ ; aucune classe isolée ✓.

Conclusion. Le diagramme obtenu modélise fidèlement les quatre règles de gestion : il

pourra servir de base à la création des tables de la base de données du lycée (chaque classe devenant une table, chaque association un lien par clé étrangère).

9.7 Méthode 7 : Produire un diagramme d'objets à partir d'un diagramme de classes

MÉTHODE Dériver un diagramme d'objets cohérent

1. Choisir un instant précis (ex. « au 15 mars 2025 ») : le diagramme d'objets est un cliché du système.
2. Instancier chaque classe utile : pour chaque objet, rectangle à deux compartiments, en-tête `nom` : `Classe` souligné, valeurs d'attributs conformes aux types.
3. Matérialiser les liens : pour chaque association du diagramme de classes, trace une ligne (un **lien**) entre les objets concernés — sans multiplicités ni losanges : un lien relie des objets précis.
4. Vérifier la conformité : chaque lien doit respecter les multiplicités du diagramme de classes (un objet ne peut pas être lié à plus d'objets que la multiplicité maximale ne l'autorise) ; tout attribut renseigné doit exister dans la classe.

Réflexe Bac : le diagramme d'objets sert à valider le diagramme de classes — si tu ne peux pas dessiner un exemple concret conforme, c'est que le diagramme de classes contient une erreur.

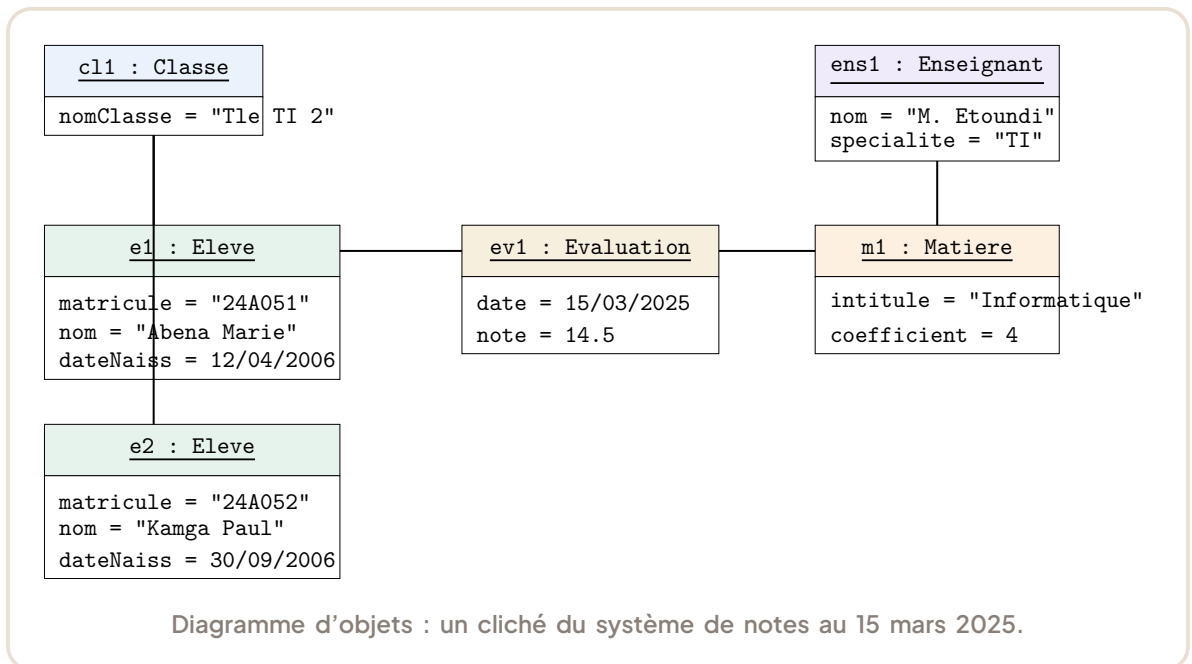
EXERCICE RÉSOLU 14 Diagramme d'objets du système de gestion de notes

À partir du diagramme de classes de l'exercice précédent, représenter un diagramme d'objets décrivant la situation suivante : la classe « Tle TI 2 » regroupe les élèves `e1` (matricule « 24A051 », nom « Abena Marie », née le 12/04/2006) et `e2` (matricule « 24A052 », nom « Kamga Paul », né le 30/09/2006) ; l'évaluation `ev1` du 15/03/2025 (note 14,5) concerne `e1` et la matière `m1` « Informatique » (coefficient 4), enseignée par l'enseignant `ens1` « M. Etoundi », spécialité « TI ».

SOLUTION

Étape 1 — Instanciation. Six objets : `cl1` : Classe, `e1` : Eleve, `e2` : Eleve, `ev1` : Evaluation, `m1` : Matiere, `ens1` : Enseignant. En-têtes soulignés, valeurs typées (la note 14.5 est bien un réel, le coefficient 4 un entier).

Étape 2 — Liens. `cl1—e1`, `cl1—e2` (association Classe–Eleve) ; `e1—ev1` (Eleve–Evaluation) ; `ev1—m1` (Matiere–Evaluation) ; `ens1—m1` (Enseignant–Matiere).



Étape 3 — Contrôle de conformité. *cl1* est liée à deux élèves : autorisé par 1..* ✓. *e1* et *e2* sont chacun liés à une seule classe : conforme à 1 ✓. *ev1* est liée à un seul élève et une seule matière : conforme aux 1 ✓. *e2* n'a aucune évaluation : autorisé par 0..* ✓.

Conclusion. Ce diagramme d'objets, conforme en tout point au diagramme de classes, valide la modélisation : les multiplicités choisies permettent bien de décrire la situation réelle du 15 mars 2025.

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Inverser les multiplicités. La multiplicité placée du côté de B décrit combien de B sont liés à un A. Écrire 1..* côté *Classe* parce que « une classe a plusieurs élèves » est l'erreur la plus fréquente : applique toujours la lecture croisée et relis les deux phrases.
2. Orienter la flèche d'héritage dans le mauvais sens. Le triangle blanc pointe vers la super-classe (la classe mère), jamais vers la sous-classe. Autre faute associée : recopier dans la sous-classe des attributs déjà hérités — l'héritage sert précisément à les factoriser.
3. Confondre agrégation et composition. Le seul critère fiable est le cycle de vie : la partie survit-elle à la destruction du tout ? Oui ⇒ losange vide (agrégation) ; non ⇒ losange plein (composition). Et dans les deux cas, le losange se place du côté du contenant.
4. Oublier visibilité et types. Une classe sans signes +/-/#, sans types d'attributs, ou avec des méthodes sans paramètres typés ni type de retour, est une classe incom-

plète : elle perd des points même si la structure est bonne. Pense aussi à mettre les attributs en privé (encapsulation).

5. Dessiner un diagramme d'objets comme un diagramme de classes. Trois fautes typiques : oublier de souligner l'en-tête `nom : Classe` ; ajouter un compartiment de méthodes (un objet n'en a pas) ; faire figurer des multiplicités ou des losanges sur les liens (un lien relie des objets précis, sans cardinalité). Enfin, vérifie que chaque valeur respecte le type déclaré dans la classe.

10 EXERCICES

10.1 Je vérifie mes connaissances

EXERCICE 7 QCM : les bases d'UML

Pour chaque question, une seule réponse est exacte. Entoure la bonne lettre.

1. UML signifie :
 - (a) Universal Modeling Language
 - (b) Unified Modeling Language
 - (c) Unified Machine Language
2. Dans un diagramme de classes, un attribut privé est précédé du symbole :
 - (a) +
 - (b) #
 - (c) -
3. La relation d'héritage se représente par :
 - (a) une flèche à trait plein et pointe triangulaire vide, dirigée vers la classe mère ;
 - (b) une flèche à trait plein et pointe triangulaire pleine, dirigée vers la classe fille ;
 - (c) un losange plein placé du côté de la classe mère.
4. La composition se représente par :
 - (a) un losange vide du côté du tout ;
 - (b) un losange plein du côté du tout ;
 - (c) un losange plein du côté de la partie.

EXERCICE 8 Vrai ou faux — justifie ta réponse

Réponds par vrai ou faux, puis justifie en une ou deux phrases.

1. Une composition se dessine avec un losange vide.
2. La multiplicité se lit du côté de la classe de départ de l'association.
3. Une classe fille recopie physiquement (par copie de code) les attributs de sa classe mère.
4. Dans un diagramme d'objets, le nom d'un objet est souligné.

5. La multiplicité 1..* signifie « au moins un ».

EXERCICE 9 Définitions

Définis en deux ou trois phrases chacun des termes suivants, en donnant pour chacun un exemple tiré d'un système d'information camerounais (école, pharmacie, mobile money, transport) :

1. une **classe** ;
2. un **objet** (ou instance) ;
3. la **multiplicité** d'une association ;
4. la **visibilité** d'un attribut ou d'une méthode.

EXERCICE 10 Lecture de multiplicités

Soit l'association suivante entre deux classes d'un système de gestion de lycée : un Professeur est relié à la classe Matière par l'association enseigne, avec la multiplicité 1..* du côté de Matière et 0..1 du côté de Professeur.

1. Combien de matières un professeur peut-il enseigner, au minimum et au maximum ?
2. Une matière peut-elle exister sans professeur associé ? Justifie.
3. Traduis l'ensemble de l'association par deux phrases en français courant.

10.2 Je m'entraîne

EXERCICE 11 Modélisation : la classe CompteMobileMoney

Une startup de Douala développe une application de simulation de portefeuille mobile. On te demande de modéliser la classe CompteMobileMoney possédant :

- les attributs privés `numero` (chaîne de caractères) et `solde` (réel, en FCFA) ;
 - les méthodes publiques `depot(montant : Reel)`, `retrait(montant : Reel) : Booleen` (retourne vrai si le retrait est possible) et `consulterSolde() : Reel`.
1. Représente cette classe en UML, avec les trois compartiments, les visibilités, les types des attributs, les paramètres et les types de retour des méthodes.
 2. Représente ensuite, dans un diagramme d'objets, deux objets clients MTN MoMo : `c1` de numéro « 677123456 » avec un solde de 45250 FCFA, et `c2` de numéro « 650987654 »

» avec un solde de 120000 FCFA.

EXERCICE 12 Chasse aux erreurs

Le diagramme de classes ci-dessous (décrit textuellement) a été dessiné par un élève pressé et contient cinq erreurs de notation UML. Identifie chaque erreur et donne la notation correcte.

- La classe `Eleve` hérite de la classe `Personne`, mais la flèche d'héritage part de `Personne` et pointe vers `Eleve`.
- L'objet `e1 : Eleve` est dessiné dans un rectangle à trois compartiments et son nom n'est pas souligné.
- La composition entre `Lycee` (le tout) et `Salle` (la partie) est dessinée avec le losange plein placé du côté de `Salle`.
- La méthode `calculerMoyenne()` de la classe `Eleve`, censée retourner un réel, est écrite sans type de retour.
- L'association fréquente entre `Eleve` et `Lycee` porte la multiplicité 1 du côté de `Eleve` et * du côté de `Lycee`, alors que l'énoncé dit : « un élève fréquente un seul lycée ; un lycée accueille plusieurs élèves ».

Présente ta réponse sous la forme d'un tableau à trois colonnes : Erreur repérée / Pourquoi c'est faux / Notation correcte.

EXERCICE 13 Agrégation ou composition ?

Pour chacun des couples de classes suivants, indique s'il s'agit d'une agrégation ou d'une composition, justifie ton choix en une phrase, puis précise de quel côté se place le losange et s'il est vide ou plein.

1. `EquipeFootball` / `Joueur` (un joueur peut exister sans équipe et changer de club).
2. `Maison` / `Piece` (une pièce n'existe pas en dehors de sa maison ; si la maison est détruite, les pièces disparaissent).
3. `Universite` / `Etudiant`.
4. `Commande` / `LigneCommande`.

EXERCICE 14 Question de synthèse

Explique en cinq lignes maximum la différence entre agrégation et composition, en t'ap-

puvant sur les deux exemples « Équipe de football / Joueur » et « Maison / Pièce ». Ta réponse doit mentionner : le sens du losange (vide ou plein), la notion de cycle de vie (durée d'existence de la partie par rapport au tout) et le critère qui permet de choisir entre les deux relations lors d'une modélisation.

10.3 Je me prépare au Bac

EXERCICE 15 Type Bac — Gestion d'une pharmacie à Yaoundé

La pharmacie « Les Lauriers » de Yaoundé souhaite informatiser la gestion de ses ventes sur ordonnance. L'analyse du domaine a produit les informations suivantes :

- Un client est caractérisé par son nom, son numéro de téléphone et sa date de naissance.
- Un médicament possède une référence, un libellé, un prix unitaire (en FCFA) et une quantité en stock.
- Un pharmacien (matricule, nom, grade) délivre les médicaments.
- Une ordonnance (numéro, date d'émission) est prescrite à un client et concerne un ou plusieurs médicaments ; elle est validée par exactement un pharmacien. Un médicament peut figurer sur plusieurs ordonnances ou sur aucune.

Le diagramme de classes ci-dessous a été partiellement effacé : les multiplicités, certaines visibilité et certains types de retour sont manquants.

Client	
- nom : Chaîne	- telephone : Chaîne
- dateNaissance : Date	
+ acheter()	
Médicament	
- reference : Chaîne	- libelle : Chaîne
- prixUnitaire : Reel	- qteStock : Entier
+ estDisponible() : ???	
Ordonnance	
- numero : Chaîne	- dateEmission : Date
+ ajouterMédicament(m : Médicament)	

Pharmacien

- matricule : Chaine - nom : Chaine - grade : Chaine

+ valider(o : Ordonnance) : ???

1. Définis les termes classe et objet, puis donne un exemple d'objet de la classe `Medicament` avec des valeurs réalistes.
2. Explique la signification du symbole « - » placé devant les attributs. Pourquoi est-il recommandé de rendre les attributs privés ?
3. Complète le type de retour manquant de la méthode `estDisponible()` et justifie ton choix.
4. Dessine le diagramme de classes complet (quatre classes) en plaçant les associations et toutes les multiplicités, conformément à l'énoncé :
 - association est prescrite à entre `Ordonnance` et `Client` ;
 - association concerne entre `Ordonnance` et `Medicament` ;
 - association valide entre `Pharmacien` et `Ordonnance`.
5. Le directeur précise : « une ordonnance est détruite si le dossier du client est supprimé du système ; elle ne peut exister sans lui ». Quelle relation (agrégation ou composition) faut-il alors utiliser entre `Client` et `Ordonnance` ? Justifie et précise la notation graphique.

EXERCICE 16 Type Bac — Plateforme de covoiturage Douala–Yaoundé

Une startup camerounaise lance une application de covoiturage interurbain. Son diagramme de classes comporte quatre classes :

- `Conducteur` (attributs : nom, telephone ; méthode : `publierTrajet()`) ;
- `Passager` (attributs : nom, telephone ; méthode : `reserver()`) ;
- `Trajet` (attributs : villeDepart, villeArrivee, dateDepart, prixPlace) ;
- `Vehicule` (attributs : immatriculation, marque, nbPlaces).

Les associations du diagramme sont les suivantes :

- un conducteur propose de zéro à plusieurs trajets ; un trajet est proposé par exactement un conducteur ;
- un passager réserve de zéro à plusieurs trajets ; un trajet peut être réservé par zéro à plusieurs passagers ;
- un conducteur conduit exactement un véhicule ; un véhicule est conduit par un seul conducteur ;

- un véhicule est une composition du profil du conducteur : si le compte du conducteur est supprimé, la fiche du véhicule disparaît avec lui.
1. Rappelle le rôle d'un diagramme de classes dans la modélisation d'un système d'information.
 2. Dessine le diagramme de classes complet de cette plateforme : classes avec leurs compartiments, associations nommées, multiplicités, et losange de composition correctement placé.
 3. On considère la situation réelle suivante : le conducteur `cond1` (M. Etoundi, 699001122) conduit le véhicule `v1` (immatriculation « LT 452 AB », marque Toyota, 4 places) et propose le trajet `t1` (Douala – Yaoundé, départ le 15/03/2025, 3500 FCFA la place). Deux passagers ont réservé ce trajet : `p1` (Aminatou, 677334455) et `p2` (Boris, 655667788). Produis le diagramme d'objets correspondant à cette situation, en respectant la notation UML des instances.
 4. Explique la différence entre le diagramme de classes de la question 2 et le diagramme d'objets de la question 3 (niveau d'abstraction, contenu, notation).
 5. Le véhicule `v1` est déclaré en composition avec `Conducteur`. Un collègue affirme qu'il aurait fallu une agrégation. Donne un argument technique justifiant le choix de la composition dans ce contexte précis.

EXERCICE 17 Type Bac — Système d'information d'un lycée de Bafoussam

Le proviseur d'un lycée de Bafoussam te confie la modélisation structurelle du système d'information de l'établissement. Après interviews, tu recueilles les faits suivants :

- Le lycée compte des personnels. Un personnel possède un matricule, un nom et un numéro de téléphone. Il existe deux catégories de personnels : les enseignants (qui ont en plus une spécialité et une méthode `saisirNotes()`) et les personnels administratifs (qui ont en plus un service et une méthode `editerBulletin()`).
 - Un élève (matricule, nom, dateNaissance) est inscrit dans exactement une classe (nomClasse, niveau, salle) ; une classe regroupe de 1 à 80 élèves.
 - Un enseignant enseigne dans une ou plusieurs classes ; une classe est encadrée par plusieurs enseignants (au moins un).
 - Une classe possède des séquences d'évaluation ; une séquence n'a aucune existence en dehors de la classe qui la possède.
1. Dessine le diagramme de classes complet (cinq classes au maximum : `Personnel`, `Enseignant`, `PersonnelAdministratif`, `Eleve`, `Classe` ; la classe `Sequence` sera ajoutée à la question 4) en faisant apparaître :
 - la relation d'héritage entre les personnels, avec la flèche correctement orientée ;

- les associations et leurs multiplicités.
2. Explique, en t'appuyant sur cet exemple, ce qu'est l'héritage et cite deux avantages de cette relation pour le concepteur (réutilisation, factorisation).
 3. Un élève du lycée, `e11` (matricule « LYCBF-2024-017 », nom « Kamga Larissa », née le 02/05/2007), est inscrite en classe `c11` (« Terminale TI 1 », niveau « Terminale », salle « TI2 »). Représente ces deux objets et leur lien dans un diagramme d'objets.
 4. Ajoute la classe `Sequence` au diagramme en choisissant entre agrégation et composition, en justifiant ton choix à partir de l'énoncé, et en précisant la notation graphique utilisée.
 5. La méthode `saisirNotes()` de la classe `Enseignant` doit retourner vrai si la saisie a réussi, faux sinon. Écris sa signature UML complète (visibilité publique, nom, type de retour).

11 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : les bases d'UML

1. Réponse b. UML signifie Unified Modeling Language (langage de modélisation unifié), né de la fusion des méthodes de Booch, Rumbaugh (OMT) et Jacobson (OOSE) au milieu des années 1990, puis normalisé par l'OMG en 1997.
2. Réponse c. Le symbole - désigne la visibilité **privée**. Rappel : + = public, # = protégé, - = privé.
3. Réponse a. L'héritage (généralisation) se représente par une flèche à trait plein terminée par un triangle vide, partant de la classe fille et pointant vers la classe mère. Le triangle plein n'existe pas en UML pour l'héritage, et le losange plein désigne la composition, pas l'héritage.
4. Réponse b. La composition se représente par un losange plein placé du côté du tout (le composite). Le losange vide correspond à l'agrégation ; le losange n'est jamais placé du côté de la partie.

CORRIGÉ Exercice 2 — Vrai ou faux

1. Faux. Une composition se dessine avec un losange plein (noir), placé du côté du tout. Le losange vide est réservé à l'agrégation, où la partie peut exister indépendamment du tout.
2. Faux. La multiplicité se lit du côté de la classe d'arrivée (en face de la classe dont on parle). Ainsi, la multiplicité placée près de B indique combien d'instances de B sont associées à une instance de A.
3. Faux. La classe fille hérite des attributs et méthodes de la classe mère sans aucune copie de code : elle y accède automatiquement par le mécanisme d'héritage. Recopier le code serait une duplication source d'incohérences.
4. Vrai. Dans un diagramme d'objets, le nom d'une instance s'écrit souligné, sous la forme `nomObjet : NomClasse` (par exemple e1 : Eleve), ce qui distingue visuellement un objet d'une classe.
5. Vrai. `1..*` signifie « au moins un » : minimum 1, maximum indéterminé (plusieurs). À ne pas confondre avec `0..*` (ou `*`), qui signifie « zéro ou plusieurs ».

CORRIGÉ Exercice 3 — Définitions

1. **Classe.** Une classe est un modèle (un moule) décrivant un ensemble d'objets partageant les mêmes attributs (données) et les mêmes méthodes (comportements). Elle définit la structure commune sans contenir de valeurs concrètes. Exemple : la classe `CompteMobileMoney` décrit tout compte MoMo (numéro, solde, dépôt, retrait) sans désigner un compte particulier.
2. **Objet (instance).** Un objet est une réalisation concrète d'une classe : il possède des valeurs précises pour chaque attribut défini par sa classe. On dit qu'il instancie la classe. Exemple : le compte `c1` de numéro « 677123456 » et de solde 45250 FCFA est un objet (une instance) de la classe `CompteMobileMoney`.
3. **Multiplicité.** La multiplicité d'une association indique, pour une instance d'une classe, le nombre minimal et maximal d'instances de l'autre classe auxquelles elle peut être reliée. Elle s'écrit `1..0..1`, `1..*`, `*`, etc. Exemple : dans une pharmacie, une ordonnance concerne `1..*` médicaments (au moins un), et un médicament figure sur `0..*` ordonnances.
4. **Visibilité.** La visibilité précise qui peut accéder à un attribut ou à une méthode : `+` public (accessible partout), `-` privé (accessible uniquement à l'intérieur de la classe), `#` protégé (accessible dans la classe et ses sous-classes). Exemple : le `solde` d'un compte mobile money est privé : on ne peut le modifier que via les méthodes publiques `depot()` et `retrait()`, ce qui protège l'intégrité des données.

CORRIGÉ Exercice 4 — Lecture de multiplicités

Rappel de la règle de lecture : la multiplicité placée en face d'une classe indique combien d'instances de cette classe sont liées à une instance de l'autre classe.

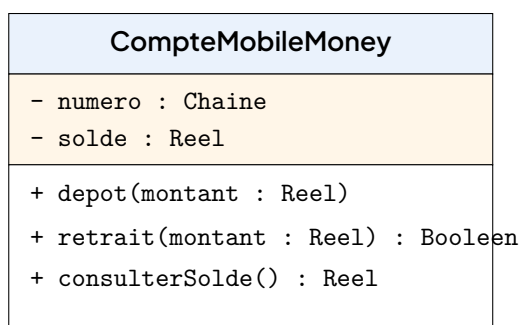
1. La multiplicité `1..*` est du côté de `Matiere`. Un professeur enseigne donc au minimum 1 matière et au maximum plusieurs (nombre indéterminé).
2. Oui. La multiplicité `0..1` du côté de `Professeur` signifie qu'une matière est enseignée par zéro ou un professeur : une matière peut exister sans professeur associé (par exemple une matière nouvellement créée, pas encore attribuée).
3. Traduction en français courant :
 - « Un professeur enseigne au moins une matière, éventuellement plusieurs. »
 - « Une matière est enseignée par au plus un professeur, et peut n'être enseignée par aucun. »

ATTENTION Piège classique

L'erreur la plus fréquente au Bac est de lire la multiplicité du côté de la classe de départ. Retiens : on lit la multiplicité en face, c'est-à-dire du côté de la classe cible.

CORRIGÉ Exercice 5 — Modélisation : la classe CompteMobileMoney

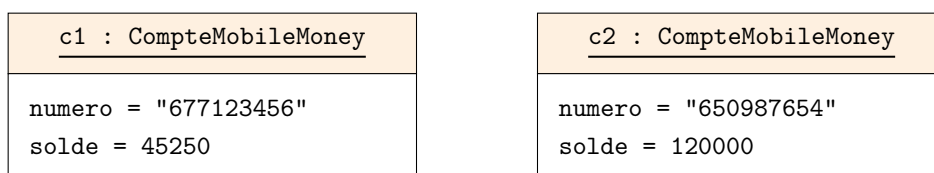
1. Diagramme de classes. Les attributs sont précédés de - (privé), les méthodes de + (public), avec paramètres typés et types de retour indiqués.



La classe `CompteMobileMoney` : trois compartiments (nom, attributs, méthodes).

Points de rigueur : `retrait` possède un paramètre typé `montant : Reel` et un type de retour `Booleen`; `depot` ne retourne rien (pas de type de retour); `consulterSolde` n'a pas de paramètre mais retourne un `Reel`.

2. Diagramme d'objets. Chaque objet est un rectangle dont le titre souligné a la forme `nomObjet : Classe`, suivi des attributs avec leurs valeurs.



Deux instances de `CompteMobileMoney` : noms soulignés, attributs valorisés, pas de compartiment méthodes.

ATTENTION Points de vigilance

Dans un diagramme d'objets : (1) le nom est souligné ; (2) on écrit `attribut = valeur` et non `attribut : type`; (3) on ne fait pas figurer les méthodes, car elles sont identiques pour toutes les instances de la classe.

CORRIGÉ Exercice 6 — Chasse aux erreurs

Erreur repérée	Pourquoi c'est faux	Notation correcte
Flèche d'héritage de <code>Personne</code> vers <code>Eleve</code>	La flèche d'héritage part toujours de la classe fille et pointe vers la classe mère (triangle vide)	Flèche à triangle vide de <code>Eleve</code> vers <code>Personne</code>
Objet <code>e1 : Eleve</code> en trois compartiments, nom non souligné	Un objet se dessine avec deux compartiments (nom souligné + attributs valorisés); les méthodes n'y figurent pas	Rectangle avec <u><code>e1 : Eleve</code></u> souligné, puis <code>attribut = valeur</code>
Losange plein du côté de <code>Salle</code>	En composition, le losange plein se place du côté du tout (le composite), jamais du côté de la partie	Losange plein du côté de <code>Lycee</code>
<code>calculerMoyenne()</code> sans type de retour	Toute méthode qui retourne une valeur doit déclarer son type de retour après les deux-points	+ <code>calculerMoyenne()</code> : <code>Reel</code>
Multiplicités inversées (1 côté <code>Eleve</code> , * côté <code>Lycee</code>)	La multiplicité se lit en face : un élève fréquente 1 lycée ⇒ 1 du côté de <code>Lycee</code> ; un lycée accueille plusieurs élèves ⇒ * du côté d' <code>Eleve</code>	* côté <code>Eleve</code> , 1 côté <code>Lycee</code>

CORRIGÉ Exercice 7 — Agrégation ou composition ?

1. `EquipeFootball` / `Joueur` : agrégation. Un joueur existe indépendamment de son équipe (il peut être sans club ou transféré) : les cycles de vie sont indépendants. Losange vide du côté d'`EquipeFootball` (le tout).
2. `Maison` / `Piece` : composition. Une pièce n'existe pas hors de sa maison et disparaît si la maison est détruite : la partie partage le cycle de vie du tout. Losange plein du côté de `Maison`.
3. `Universite` / `Etudiant` : agrégation. Un étudiant continue d'exister s'il quitte l'univer-

sité ou si celle-ci ferme ; il peut même changer d'université. Losange vide du côté d'Université.

4. `Commande / LigneCommande` : composition. Une ligne de commande n'a de sens que dans sa commande ; supprimer la commande supprime ses lignes. Losange plein du côté de `Commande`.

MÉTHODE Critère de décision

Pose la question : « si le tout est détruit, la partie peut-elle continuer à exister seule ? » Si oui $\Rightarrow$ agrégation (losange vide) ; si non $\Rightarrow$ composition (losange plein). Dans les deux cas, le losange est du côté du tout.

CORRIGÉ Exercice 8 — Question de synthèse

L'agrégation et la composition expriment toutes deux une relation « tout / partie », mais la composition est une agrégation forte. Dans l'agrégation (losange vide du côté du tout), la partie a un cycle de vie indépendant : un joueur survit à la dissolution de son équipe de football et peut changer de club. Dans la composition (losange plein du côté du tout), la partie partage le cycle de vie du tout : une pièce n'existe pas sans sa maison et disparaît si celle-ci est détruite. Le critère de choix est donc la dépendance existentielle de la partie vis-à-vis du tout : destruction conjointe $\Rightarrow$ composition, existence autonome $\Rightarrow$ agrégation.

CORRIGÉ Exercice 9 — Type Bac : pharmacie « Les Lauriers »

1. Une classe est un modèle abstrait décrivant les attributs et les méthodes communs à un ensemble d'objets ; un objet est une instance concrète d'une classe, avec des valeurs précises pour ses attributs. Exemple d'objet de `Medicament` : `m1 : Medicament` avec `reference = "PARA500"`, `libelle = "Paracétamol 500 mg"`, `prixUnitaire = 500`, `qteStock = 240`.
2. Le symbole `-` signifie que l'attribut est privé : il n'est accessible que depuis les méthodes de la classe elle-même. Rendre les attributs privés (principe d'encapsulation) protège l'intégrité des données : par exemple, on empêche de fixer directement `qteStock` à une valeur négative ; toute modification passe par des méthodes publiques qui contrôlent la validité.
3. `+ estDisponible() : Booleen`. La méthode répond à la question « le médicament est-il disponible ? » : elle retourne vrai si `qteStock > 0`, faux sinon. Le type de retour est donc `Booleen`. (On accepterait difficilement `Entier` : ce serait alors une méthode `quantiteDisponible()`.)

4. Multiplicités déduites de l'énoncé :

- **est prescrite à** : une ordonnance est prescrite à exactement un client $\Rightarrow$ 1 côté Client; un client peut recevoir plusieurs ordonnances ou aucune $\Rightarrow$ 0..* côté Ordonnance.
- **concerne** : une ordonnance concerne un ou plusieurs médicaments $\Rightarrow$ 1..* côté Medicament; un médicament figure sur plusieurs ordonnances ou aucune $\Rightarrow$ 0..* côté Ordonnance.
- **valide** : une ordonnance est validée par exactement un pharmacien $\Rightarrow$ 1 côté Pharmacien; un pharmacien valide zéro ou plusieurs ordonnances $\Rightarrow$ 0..* côté Ordonnance.

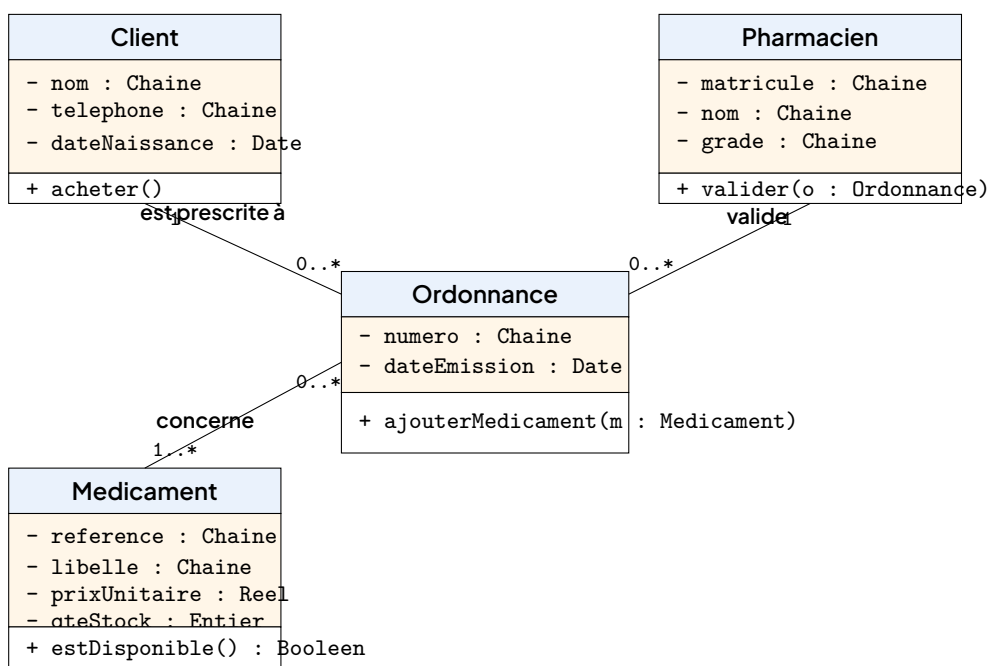


Diagramme de classes de la pharmacie (associations et multiplicités).

5. Il faut utiliser une composition : l'énoncé précise que l'ordonnance ne peut exister sans le client et est détruite avec son dossier — la partie partage le cycle de vie du tout. Notation : losange plein (noir) placé du côté de Client (le tout), relié à Ordonnance (la partie), avec en général la multiplicité 1 côté Client et 0..* côté Ordonnance.

Barème indicatif (sur 10) : définitions + exemple d'objet : 2 pts ; visibilité et encapsulation : 1,5 pt ; type de retour justifié : 1,5 pt ; diagramme complet (3 associations, 6 multiplicités correctes) : 3 pts ; composition justifiée + notation : 2 pts.

Points de vigilance : la justification de la composition doit citer explicitement la destruction conjointe (cycle de vie lié) ; chaque multiplicité doit être justifiée par une phrase de l'énoncé ; ne pas oublier les noms des associations.

CORRIGÉ Exercice 10 — Type Bac : covoiturage Douala–Yaoundé

1. Le diagramme de classes décrit la structure statique du système d'information : les types d'objets manipulés (classes), leurs données (attributs), leurs comportements (méthodes) et les relations qui les lient (associations, héritage, agrégation, composition). Il sert de schéma de référence pour la conception de la base de données et le développement de l'application.

2. Multiplicités : propose : 1 côté Conducteur, 0..* côté Trajet ; réserve : 0..* des deux côtés ; conduit : 1 des deux côtés, avec losange plein du côté de Conducteur.

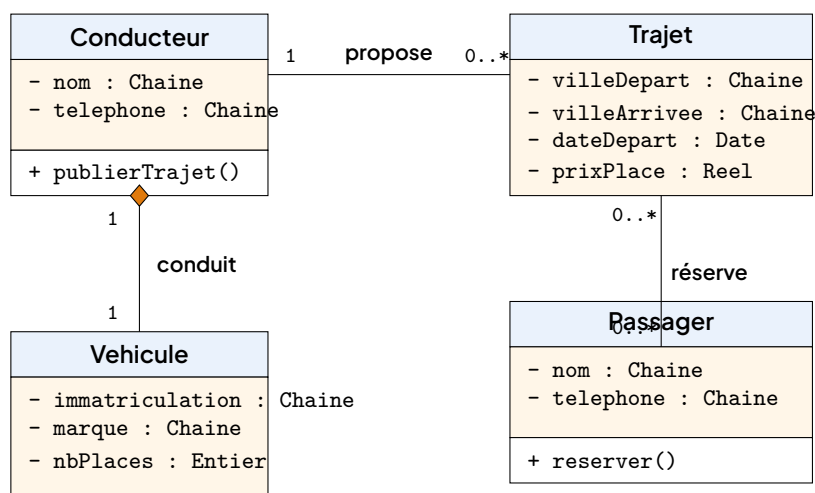


Diagramme de classes de la plateforme de covoiturage (losange plein côté Conducteur).

3. Diagramme d'objets : noms soulignés, attributs valorisés, liens matérialisés par des traits simples (sans losange ni multiplicité).

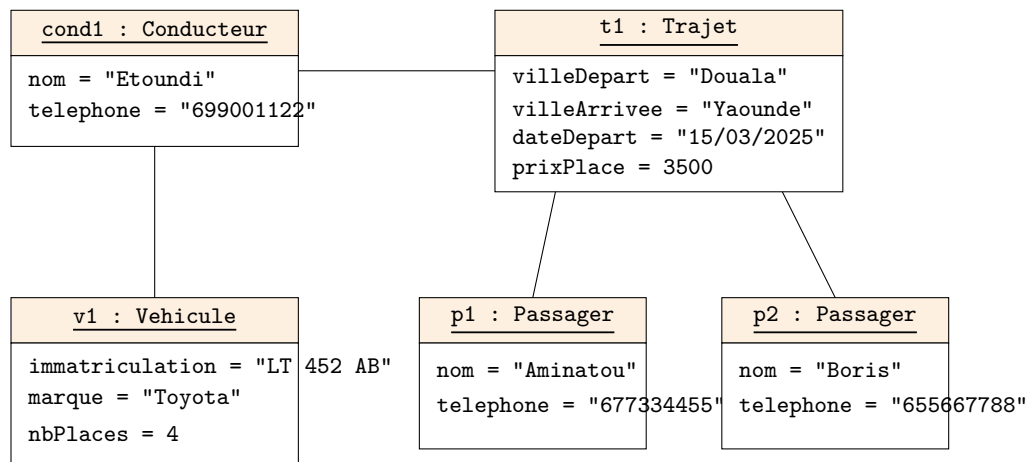


Diagramme d'objets : cond1 conduit v1 et propose t1, réservé par p1 et p2.

4. Le diagramme de classes se situe au niveau abstrait et général : il décrit des types d'objets, avec les types des attributs (`nom : Chaîne`), les méthodes, les multiplicités et les relations structurelles (losanges, flèches). Le diagramme d'objets se situe au niveau concret : il représente des instances à un instant donné, avec des valeurs (`nom = "Etoundi"`), des noms soulignés, des liens simples (sans multiplicité ni losange) et sans compartiment méthodes. Le diagramme d'objets est une « photographie » d'une situation particulière conforme au diagramme de classes.

5. Argument pour la composition : dans cette application, la fiche `Vehicule` n'existe que comme donnée du profil du conducteur ; elle n'a aucune existence autonome dans le système (on ne référence pas un véhicule « orphelin »). La suppression du compte conducteur entraîne donc la suppression de la fiche véhicule : les cycles de vie sont liés, ce qui est exactement la sémantique de la composition (losange plein). Une agrégation impliquerait que le véhicule puisse exister dans le système indépendamment de tout conducteur, ce qui contredit l'énoncé.

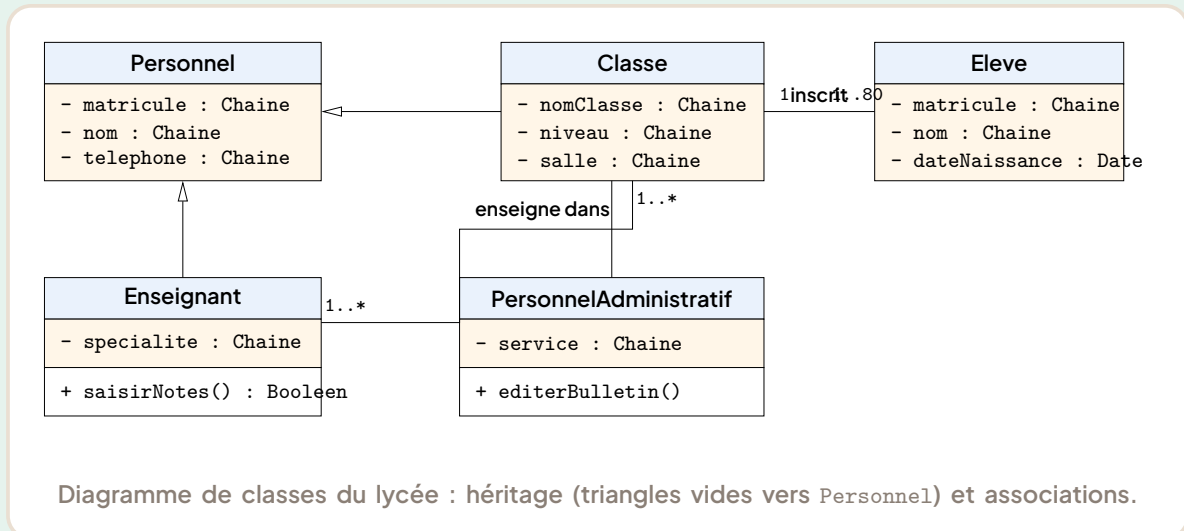
Barème indicatif (sur 10) : rôle du diagramme : 1 pt ; diagramme de classes (4 classes, 3 associations, multiplicités, losange plein bien placé) : 4 pts ; diagramme d'objets conforme (noms soulignés, valeurs, liens) : 3 pts ; comparaison classes/objets : 1 pt ; argument composition : 1 pt.

Points de vigilance : dans le diagramme d'objets, ne jamais faire figurer les méthodes ni les multiplicités ; vérifier que chaque objet respecte la structure de sa classe ; le losange plein doit être rempli et du côté de `Conducteur`.

CORRIGÉ Exercice 11 — Type Bac : lycée de Bafoussam

1. Diagramme de classes : `Enseignant` et `PersonnelAdministratif` héritent de `Personnel`

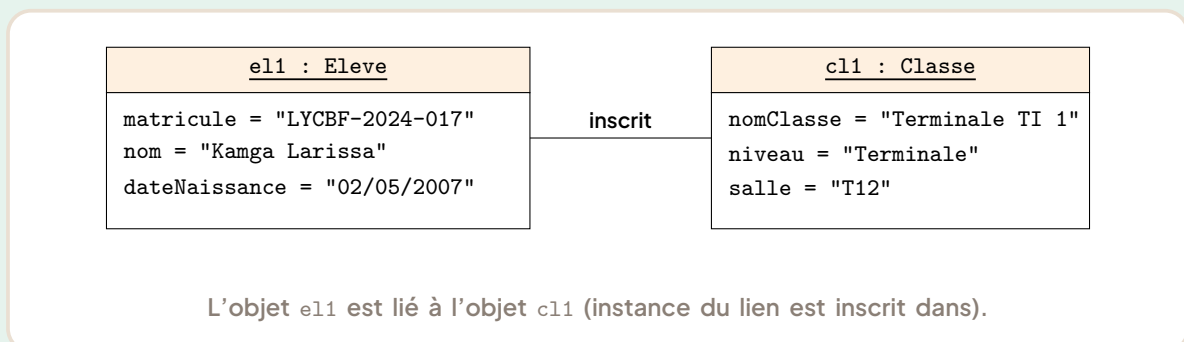
(flèches à triangle vide dirigées vers `Personnel`). Multiplicités : est inscrit dans : 1..80 côté `Eleve`, 1 côté `Classe`; enseigne dans : 1..* des deux côtés.



2. L'héritage est une relation « est un » entre une classe mère (`Personnel`) et des classes filles (`Enseignant`, `PersonnelAdministratif`) qui acquièrent automatiquement tous les attributs et méthodes de la mère, tout en ajoutant leurs spécificités. Avantages pour le concepteur :

- Réutilisation / factorisation : `matricule`, `nom` et `telephone` ne sont déclarés qu'une seule fois dans `Personnel` au lieu d'être recopiés dans chaque catégorie ;
- Maintenance facilitée : ajouter un attribut commun (par exemple `email`) dans `Personnel` le propage automatiquement aux deux classes filles, sans risque d'oubli ni d'incohérence.

3. Diagramme d'objets :



4. L'énoncé dit qu'une séquence « n'a aucune existence en dehors de la classe qui la possède » : c'est une composition. Notation : losange plein du côté de `Classe` (le tout), relié à `Sequence` (la partie), avec la multiplicité 1 côté `Classe` et 1..* côté `Sequence` (une classe possède plusieurs séquences, chaque séquence appartient à exactement une classe). La suppression de la classe entraîne la suppression de ses séquences.

5. Signature UML complète :

+ saisirNotes() : Booleen

Le + indique la visibilité publique, les parenthèses vides l'absence de paramètre, et : Booleen le type de retour (vrai si la saisie réussit, faux sinon).

Barème indicatif (sur 10) : diagramme de classes (héritage correctement orienté : 2 pts ; associations et multiplicités : 2 pts) ; explication de l'héritage + 2 avantages : 2 pts ; diagramme d'objets conforme : 2 pts ; composition justifiée + notation : 1 pt ; signature de méthode : 1 pt.

Points de vigilance : la flèche d'héritage doit partir des classes filles vers la classe mère avec un triangle vide ; la multiplicité 1..80 (et non *) traduit l'effectif borné de l'énoncé ; dans le diagramme d'objets, noms soulignés et valeurs (pas de types) ; la justification de la composition doit citer la dépendance d'existence de la partie.

12 FICHE BILAN



Carte mentale de la leçon : modélisation structurelle avec UML.

FICHE BILAN L'essentiel à connaître pour l'examen

Définitions clés

- **UML** (Unified Modeling Language) : langage de modélisation graphique normalisé, in-

dépendant de tout langage de programmation, utilisé pour analyser et concevoir un système d'information.

- **Diagramme de classes** : diagramme structurel (statique) qui décrit les classes du système, leurs caractéristiques et leurs relations. C'est le diagramme le plus important d'UML.
- **Diagramme d'objets** : photographie du système à un instant t ; il représente des instances (objets) des classes avec les valeurs de leurs attributs.
- **Classe** : modèle (moule) décrivant un ensemble d'objets partageant les mêmes attributs et les mêmes méthodes.
- **Objet** : instance concrète d'une classe (ex. `eleve1 : Eleve`).
- **Association** : lien structurel entre classes ; la **multiplicité** indique combien d'objets participent au lien.
- **Héritage** : relation « est un » ; la sous-classe hérite des attributs et méthodes de la super-classe.

Syntaxe UML à connaître par cœur

Notation	Signification
+	attribut ou méthode public (visible partout)
-	privé (visible uniquement dans la classe)
#	protégé (visible dans la classe et ses sous-classes)
- nom : Type	attribut avec son type (ex. - nom : String)
+ m(p : T) : R	méthode : paramètre p de type T , type de retour R
nomObjet : Classe	objet (texte souligné dans le diagramme)
1, 0..1, 1..*, *	multiplicités : un, au plus un, au moins un, plusieurs

Relations entre classes (symboles)

- Association : trait plein simple, avec nom, rôles et multiplicités aux extrémités.
- Héritage : flèche à triangle vide pointant vers la super-classe.
- Agrégation (relation « est composé de » faible) : losange vide côté contenant ; la partie peut exister sans le tout.
- Composition (agrégation forte) : losange plein ; la partie disparaît avec le tout (ex. `Commande` et `LigneCommande`).

Méthode express : construire un diagramme de classes

1. Repérer les noms du cahier des charges → classes et attributs.
2. Repérer les verbes → méthodes et associations.
3. Fixer la visibilité (attributs privés, méthodes publiques par défaut).
4. Placer les multiplicités en se posant la question dans les deux sens.

5. Identifier héritages, agrégations et compositions ; vérifier la cohérence.

À RETENIR Checklist avant le Bac

- ☐ Je sais citer le rôle des diagrammes de classes et d'objets (diagrammes structuraux/statiques).
- ☐ Je dessine une classe avec ses 3 compartiments : nom, attributs, méthodes.
- ☐ Je connais les trois visibilitées : + public, - privé, # protégé.
- ☐ J'écris correctement une méthode : visibilité, nom, paramètres typés, type de retour.
- ☐ Je représente un objet : `nom : Classe` souligné, attributs valués.
- ☐ Je distingue classe (modèle) et objet (instance) sans les confondre.
- ☐ Je place correctement les multiplicités 1, 0..1, 1..*, * aux deux bouts d'une association.
- ☐ Je trace l'héritage : flèche à triangle vide vers la super-classe.
- ☐ Je différencie agrégation (losange vide) et composition (losange plein) avec un exemple.
- ☐ Je sais produire un diagramme de classes d'au plus 5 classes à partir d'un énoncé.
- ☐ Je sais dériver le diagramme d'objets correspondant à un diagramme de classes.
- ☐ Je vérifie mon schéma : pas de classe isolée, pas d'attribut sans type, multiplicités cohérentes.



Signé • OnBuch+

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+